

Étape 6 — Recodage du bot : analyse des sources, registre de correction et architecture cible (pré-validation)

 **Aucune ligne de code n'est écrite dans ce livrable.** Neuf points bloquants et une erreur arithmétique structurelle dans le mécanisme retenu doivent être tranchés avant le codage. Voir la page **7. Points nécessitant votre validation.**

Ce que j'ai reçu, ce que je n'ai pas

Élément annoncé	Reçu	Conséquence
<code>etap45.md</code> = Étape 4, analyse de consensus	Intégral, 1 325 lignes, sections 0 à 8.8 + vérification finale	Base principale du livrable
<code>etap54.md</code> = Étape 5, formules de vente	Intégral, 437 lignes, sections 1 à 11 + conclusion	Base du côté sortie
Étapes 1, 2, 3	Absents	Reconstitués uniquement par ce que l'étape 4 en cite. Le document 3 <code>conclusions_etapes_2_et_3.md</code> était déjà livré vide à l'étape 4, et les documents 5 et 6 aussi (reconstruits en 0.1 à 0.7)
<code>ETAP4.MD</code> brut (9 988 lignes, ~25 rapports d'agents)	Absent	Je ne peux pas vérifier les citations ni recalculer les agrégats. Je prends les comptages du consensus comme donnés
PDF <code>verdict-audit-17-pages</code>	Absent	Les valeurs « PDF » sont connues seulement par la colonne « Valeur PDF » de la section 6
Méthode d'achat, journaux de sessions, erreurs identifiées, fragments de code	Absents	La porte d'entrée est reconstruite depuis le pseudocode de la section 7.3, pas depuis le code réel

CSV de sessions	Absents	Aucune mesure indépendante possible de ma part
-----------------	---------	--

Je ne considère donc **aucune** valeur de ce livrable comme vérifiée par moi. Tout ce qui suit est marqué en provenance.

0. Traitement de l'encodage

Diagnostic

Les deux fichiers présentaient un **double encodage** : du texte UTF-8 encodé une première fois en UTF-8, puis relu comme du cp1252, puis ré-encodé. Signature au niveau des octets : la séquence `C3 83 C2 A9` là où `é` (`C3 A9`) était attendu, ce qui affiche `Ã©`. Symptômes observés :

- Tous les caractères accentués doublés : `Ã©` pour `é`, `Ã¨` pour `è`, `Ã` pour `à`, `Ã´` pour `ô`.
- Guillemets français et tirets convertis : `Ã«` / `Ã»` pour `«` / `»`, `â€”` pour le tiret cadratin.
- Symboles mathématiques et grecs détruits : `ïfâ,fâ,€` pour `σ30`, `ï„` pour `τ`, `ï”` pour `Δ`, `â^` pour le signe moins typographique.
- Espaces insécables perdus : la séquence `C3 A0` (`à`) a été normalisée en `Ã` + espace ordinaire, ce qui casse la réversibilité mécanique.

Reconstitution

Réparation en un passage : lecture UTF-8, restauration des espaces insécables perdus après `Ã` et `Â`, ré-encodage cp1252 avec repli latin-1, décodage UTF-8. Résultat :

Fichier	Lignes	Caractères irrécupérables après réparation
<code>etap54.md</code> (Étape 5)	437	0
<code>etap45.md</code> (Étape 4)	1 325	8

Les 8 caractères irrécupérables

Tous les huit occupent la même position syntaxique, dans huit règles de forme `SI A [car] B ALORS refus` :

- Règle 6 et règle d'action 6 : `t0_source [car] officiel`
- Règle 7 et règle d'action 7 : `k_source [car] officiel`
- Règle 22 et règle d'action 22 : `position [car] ∅`
- Relation 8.7.2 et règle d'action 8.7.2 : `signe(m*(τ)) [car] signe(m*(τ_envoi))`

Reconstitution retenue : `≠` (**U+2260**). Justification : l'octet source est triple-mangé depuis `E2 89 A0` ; et les huit règles n'ont de sens logique qu'en inégalité (une session est refusée si la source n'est **pas** officielle, un ordre est annulé si le signe **a changé**). Aucune autre lecture ne produit une règle cohérente.

✓ **Aucune portion des deux fichiers ne reste ambiguë après réparation.** Je n'ai eu à inventer aucune information. Les seules zones marquées « reconstruit » dans ce livrable sont celles que l'étape 4 déclare elle-même reconstruites (documents 3, 5 et 6 livrés vides).

1. Analyse du fichier Étape 4 (analyse de consensus)

Résumé

Consolidation de ~25 rapports d'agents indépendants, 41 blocs d'analyse de session, ~25 fenêtres BTC 5 minutes du 18/08/2026 (08:50 à 15:05 ET), deux générations de gabarit. Le document juge les 22 fonctions indicatrices de l'étape 3, en retient 20 sous condition, en rejette 3 comme directionnelles ou inertes (T5, T12, T14), en ajoute 3 manquantes, et remplace le mécanisme économique de la stratégie.

Informations fiables (fait confirmé, comptage explicite)

Fait	Mesure	Comptage
La latence oracle vers carnet n'existe pas	Argmax médian -0,75 s ; P10 -28,25 s ; P90 0 s	27/27 mesurantes, 0/27 dans la bande postulée [+5;+60], 26/27 avec $L \leq 0$
$\sigma_{30} = 26,50 \$$ est faux	Estimateur A (incréments $1 s \times \sqrt{30}$) : médiane des médianes 7,20 \$, facteur d'erreur $\times 3,68$	29/29
La formule à 22 tests est stérile	0 entrée, conjonction complète 0/610 snapshots	19/19 agents
Invariant de complémentarité exact	<code> bid_OUI + ask_NON - 1 = 0</code>	1 025/1 025 lignes ; second invariant 1 024/1 026, écart max 1 tick
Le basis spot - oracle est un décalage de source	+42,1 \$ médian, positif sur 100 % des lignes appariées, ne se referme jamais	12/12, 5 sessions instrumentées
L'issue se verrouille après la fermeture de la fenêtre d'entrée	$\tau_{lock} = \mathbf{262 s}$; 16 à 18 % du chemin de prix posté dans les 20 % finaux	8/8
Aucun prédicteur	R^2 max 0,207 (X_{60}) ; le carnet réplique m^* à $R^2 =$ 0,904 ; $p_{brut} \leftrightarrow ask$ $R^2 =$ 0,956	9/9

Desserrer les seuils rend le bot perdant	~13 800 configurations balayées : PnL médian -2,28 \$ pour 10 \$, 94 % de réglages perdants, 2,7 % rentables au mieux	8/8
Onze champs requis sur vingt-quatre sont absents de la totalité du corpus	K, endDate, L2, position, HALT, fee_bps, order_id, fills, issue officielle, ts_src numérique, side des trades	0/41 chacun

Informations ambiguës ou contradictoires

- Deux estimateurs de σ_{30} coexistent** (A = incréments remis à l'échelle, médiane 7,20 \$, n=21 ; B = niveaux sur la fenêtre, médiane 2,25 \$, n=8), facteur ~3. L'étape 4 tranche pour A, avec une justification correcte (la variance des incréments s'additionne, c'est la loi en $\sqrt{\cdot}$). **Retenu : A, définition unique, identique entrée et sortie.**
- $k_\lambda = 0,5513$ est simultanément « confirmée » et « contredite »** (5/7 contre 2/7). L'arbitrage de l'étape 4 est bon : $0,5513 = \sqrt{3/\pi}$ est une identité algébrique confirmée par ajustement direct sur le mid (0,54 / 0,51 / 0,58), tandis que le balayage de Brier (optimum 8,00 en S1 contre 0,15 en S2, facteur 53) ne mesure pas λ mais démontre que **p n'est pas calibré**. Les deux résultats ne portent pas sur la même grandeur.
- L'écart ancrage/carnet en fin de session est « confirmé ET contredit »** (6/9 contre 3/9). Non universel : rémunérateur quand l'oracle est décroché ($D >$ frais sur 57,7 % des secondes), piège quand il oscille (écart médian **négatif**, -0,0611 et -0,0547). C'est ce qui motive la condition **$z \geq 1,0$ ET $X_{60} = 0$** .
- Contradiction frontale entre deux agents sur le plancher d'ask** : S1005 « acheter à ask $\leq 0,45$ » (142/142 gagnants) contre S1010 « ne jamais acheter sous 0,50 » (ask $< 0,50 \rightarrow 0/19$, $E[\text{PnL}/\text{part}] = -0,42$). Les deux décrivent le côté qui a gagné sur leur propre session. L'arbitrage de l'étape 4 (remplacement par un test d'espérance nette symétrique) est le seul défendable.
- Dispersion de δ_{guard} en phase milieu d'un facteur 37** (0,01803 contre 0,67669). L'étape 4 en tire correctement le refus de tout ajustement continu (les régressions à $R^2 = 1,000$ sur deux points sont surajustées) et retient une enveloppe monotone par paliers.
- La régression $e(\tau) = 0,04998 + 0,003358 \cdot \tau$ ($R^2 = 0,958$) est incompatible avec les médianes par phase du même document** : elle donne 0,654 pt à $\tau = 180$ s là où la médiane mesurée est 0,090 pt. Voir registre R39, non résolu.

Hypothèses nécessaires à la compréhension

- Que τ dans les formules désigne le temps de fenêtre issu de **endDate - 300 s**, et non l'horloge de capture relative des CSV. Le document le dit, mais aucune session ne contient **endDate**, donc **tous les seuils en secondes du livrable sont approximatifs** (registre R02).
- Que **p_ancré = $\Phi(z)$** utilise la fonction de répartition normale standard. Non écrit explicitement.
- Que le tick vaut 0,01. Le document précise lui-même que c'est **inféré du BBO, pas un champ de schéma**.
- Que **poster_passif()** place effectivement un ordre non agressif. Cette hypothèse est **fausse** telle que la porte est écrite (registre R36, point le plus grave du livrable).

Éléments importants pour la nouvelle architecture

- **Le remplacement du mécanisme économique.** La justification écrite de la stratégie (latence de propagation oracle vers carnet) est réfutée. Le remplaçant retenu est la **décroissance déterministe de $\sigma_T(\tau) = \sigma_{30} \cdot \sqrt{((300-\tau)/30)}$** , que personne ne re-cote parce que c'est un événement silencieux. C'est intellectuellement solide : c'est la seule grandeur du système connue à l'avance, et la dégressivité de la décote en devient une conséquence arithmétique et non un réglage.
- **Le déplacement de la fenêtre d'action de [60;240] vers [180;270]**, réclamé simultanément par trois familles de mesures indépendantes (mouvement résiduel de l'oracle 11,62 \$ → 1,50 \$, $\tau_{lock} = 262$ s, δ_{guard} 0,677 → 0,0157 pt).
- **Six dimensions indépendantes suffisent** au lieu de 22 tests : validité de la mesure, temps restant, détermination **z**, écart carnet/ancrage, qualité de carnet, état interne. Les redondances sont mesurées ($R^2(mid, ask) = 0,96$; $R^2(ask, N) = 0,90$; $R^2(ratio, p) = 0,90$).
- **La journalisation est le rang 1 de l'implémentation**, avant toute ligne de code de trading, avec un code de refus atomique par test (22 codes) et 24 champs + 8 horodatages.
- **M3 avant M1.** Le mécanisme d'instrumentation (5 parts, balayage de δ , objectif mesure) doit tourner avant toute cotation en taille réelle, parce que c'est le seul moyen de créer les onze champs manquants.

Éléments à confirmer avant le codage

R01, R02, R36, R37, R38, R39, R40, R41 du registre (page 1 suivante). Le plus lourd : **si **K** officiel n'est pas obtainable via l'API, l'étape 4 conclut elle-même à « arrêt du projet en l'état ».**

Liens avec les étapes 1 à 5

L'étape 4 cite l'étape 1 pour l'inventaire des 60 termes, les trous d'inventaire 1.4 n°2 et n°4, le *fail-open* accidentel de 1.3.bis §3, l'arbitrage de **PENTE_DIVISEUR** en 1.5, et le conflit de capital 5,00 \$ / 2,00 \$ de 1.6 §6. Elle déclare les conclusions des étapes 2 et 3 **livrées vides** et reconstruites depuis le PDF. L'étape 5 se déclare dérivée de l'étape 4 mais **raisonne sur une version antérieure de ses fenêtres** (registre R43).

Risques techniques et stratégiques

Risque	Nature	Gravité
Le corpus couvre une seule journée , volatilité annualisée implicite 6,9 % contre 40 à 60 % typiques pour BTC	Stratégique	Critique. Tout paramètre calibré dessus est spécifique à ce calme
3 issues indépendantes seulement. Un marqueur à 3/3 a un IC95 de Clopper-Pearson de [29,2 % ; 100 %] ; il faut 29 sessions sans faute pour rejeter $H_0(p \leq 0,90)$	Stratégique	Critique. Aucun taux du corpus n'est prédictif
Aucun PnL par sous-groupe calculable au	Technique et stratégique	Critique. Le test de non-directionnalité

niveau d'un fill, dans aucun rapport, pour aucune règle		opérationnel est inexécutable aujourd'hui
Les trois sessions du premier rapport étaient toutes DOWN : un filtre pariant systématiquement DOWN affiche un sans-faute sans rien prédire	Stratégique	Élevée. Garde permanente requise
Trois des cinq pics de gros flux (10 % des événements, 63 % du notionnel) tombent après $\tau = 240$ s, donc dans la fenêtre d'action retenue	Stratégique	Élevée. Borne la capacité
Budget de latence du bot non déterminable ; la fenêtre utile est de 2 s et la traversée de seuil dure 0,300 s	Technique	Élevée
Le carnet cesse de publier vers 273 s et le point [299;300] manque systématiquement	Technique	Moyenne, gérée par $\tau_{\text{max}} = 270$ s

2. Analyse du fichier Étape 5 (formules de vente)

Résumé

Transposition de la méthode au côté sortie. Pose la bonne question (vendre si la liquidation nette dépasse la valeur de conservation, ou si une condition de sécurité invalide la conservation), énumère 5 sorties exhaustives, définit 9 conditions V0 à V8, un ordre d'évaluation, 7 contradictions à interdire, un pseudocode normatif, et un plan de falsification. Verdict propre et honnête : **logiquement définissable, empiriquement non démontrée**, aucune vente anticipée observée dans le corpus.

Informations fiables

- L'inventaire des sorties est **exhaustif et correct** : `SELL_FILLED`, `HOLD`, `SETTLE_WIN`, `SETTLE_LOSS`, `SUSPEND_DATA`, `SUSPEND_LIQUIDITY`, `UNKNOWN`. Le refus de réduire la sortie à `SELL` / `HOLD` est le meilleur apport du document.
- La séparation `q_t = p_t` pour YES et `q_t = 1 - p_t` pour NO, et l'interdiction de déclarer une vente au bid en utilisant l'ask.
- **La comptabilité des frais**, §8.3 : réutiliser `frais_AR(ask) = 0,07*ask(1-ask) + 0,01` pour une sortie seule crée un biais de comparaison, puisque la simulation conserve jusqu'au règlement. C'est un vrai défaut de méthode, correctement identifié.

- Le cadrage en **double arrêt** (entrée et sortie sont un seul problème, pas deux filtres empilés). C'est la lecture correcte, et elle a une conséquence architecturale directe : un module de décision unique, pas deux moteurs indépendants.
- La mise en garde contre la recreation du piège T14/T15 : « un filtre apparemment prudent, mais impossible à déclencher ».
- $s \leq 0,02$ sur 538/543 instants : chiffre cohérent avec la médiane de 0,02 mesurée à l'étape 4.

Informations ambiguës ou contradictoires

Sept contradictions avec l'étape 4, toutes détaillées au registre : **R42** (politique de sortie), **R43** (fenêtres obsolètes), **R44** (seuils de fraîcheur), **R45** (borne basse de spread absente), **R46** (V1 exige un q_t calibré que l'étape 4 interdit), **R47** (V5 ressuscite la constante 0,92 supprimée), **R48** (V2 comble une lacune réelle de l'étape 4).

L'alerte de provenance §1.3 (« deux séries de chiffres pour σ_{30} ») est **résolue par l'étape 4** : ce ne sont pas deux séries contradictoires mais deux estimateurs différents, et l'étape 4 tranche pour A. L'étape 5 n'avait pas cette information.

Hypothèses nécessaires

- Que q_t soit estimable. Elle ne l'est pas : l'étape 4 démontre que p échoue au Brier par sous-groupe et l'exclut de toute porte. **V1 et V5 sont donc non implémentables en l'état**, ce que l'étape 5 pressent (« seuil désactivé ») sans en tirer la conséquence.
- Que $V^{\text{hold}}_t \approx q_t$. Approximation acceptable **seulement** si la conservation jusqu'au règlement est la politique par défaut, ce qui est justement ce que R42 met en question.
- Que des entrées existent vers $\tau \approx 122 s$. Faux après le déplacement de fenêtre.

Éléments importants pour la nouvelle architecture

- Le codage d'état à 7 valeurs, non réductible.
- L'ordre d'évaluation à 9 étapes, qui met les vetos de données et de liquidité **avant** toute logique économique.
- Les 7 contradictions à interdire, qui sont des **invariants testables** et pas des recommandations. Je les transforme en assertions d'exécution dans le module de décision.
- **SUSPEND_LIQUIDITY** plutôt que **HOLD** silencieux : le système doit dire qu'il **n'a pas pu** sortir.
- La bande morte : friction aller-retour 0,044 à 0,047 par unité au voisinage de ask = 0,50 à 0,71, soit environ 0,5 à 0,6 \$ de mouvement oracle. Toute variation plus petite ne justifie pas un aller-retour.

Éléments à confirmer avant le codage

R42 en priorité absolue : **l'étape 4 dit « sortie unique, au règlement, pas de sortie intermédiaire », l'étape 5 construit tout un moteur de sortie anticipée**. Ces deux documents ne décrivent pas le même bot. Et la réponse dépend elle-même de R36.

Risques

- **Empilement stérile**. Le document en est conscient mais V0 à V8 font 9 conditions supplémentaires par-dessus 20 conditions d'entrée : 29 conjonctions sur un corpus à 3 issues. Le risque de reproduire

exactement la stérilité de l'étape 3 est élevé.

- **Faux fills.** Sans profondeur ni `order_id`, `fill_confirmed` n'est pas calculable : V7 tel qu'écrit n'est pas évaluable.
- **Whipsaw sur V2** si `h` est trop court, et incompatibilité avec `TRAVERSEE_SEUIL` de l'étape 4 qui annule au premier tick.

3. Ce qui change tout : la porte d'entrée retenue produit un ordre agressif

🛑 C'est ma conclusion la plus importante, et elle est purement arithmétique. Elle n'est pas dans les fichiers.

La porte de la section 7.3 exige, entre autres :

```
e = |p_ancré - ask_côté|
δ = max(δ_min, δ_guard)          avec δ_min = frais_AR + 0,01
ENTRER ☒ ... et e > δ_min depuis ≥ 2 s ...
prix_limite = arrondi_tick_bas(p_ancré - δ)
poster_passif(côté, prix_limite, N, validité = 2 s)
```

Quand la porte s'ouvre, on a `p_ancré - ask > δ_min`, donc `p_ancré - δ_min > ask`, donc `prix_limite > ask`. Un ordre d'achat limite au-dessus du meilleur ask traverse le spread et **s'exécute immédiatement en taker, au ask**.

Conséquences en cascade :

1. `poster_passif()` **est un nom trompeur** : la porte garantit un ordre agressif à chaque déclenchement. Le mécanisme n'est pas de la tenue de marché.
2. **Toute la statistique de remplissage est hors sujet.** Le « taux de contact du BBO dans les 2 s » (88 % en phase milieu, 49 % en fin), la durée de vie de 2 s, le balayage de δ de M3 : ces mesures décrivent la probabilité qu'une quote passive soit touchée. Un ordre qui traverse le spread est rempli à 100 % ou rejeté, immédiatement.
3. **Le chiffre économique central du livrable est inatteignable.** L'étape 4 écrit : « PnL net attendu par événement : `N × (δ - frais_AR_pnl)` », soit `6 × (0,0157 - 0,0032) = +0,075 $`. Cette formule est celle d'une capture de spread par une quote passive remplie. Elle ne s'applique pas. Le PnL réel d'un achat au ask détenu jusqu'au règlement est `+N × (1 - ask - frais)` si le côté ancré gagne, `-N × ask` s'il perd. À `ask = 0,80` et `N = 6` : **+1,18 \$ ou -4,80 \$**.
4. **Le point mort devient un taux de réussite directionnel.** Il vaut approximativement `ask`, soit **80 % à 96 %** dans la fenêtre [180;270] où les asks du côté ancré sont hauts. Autrement dit, le mécanisme exige que `Φ(z)` soit une **probabilité calibrée** au niveau de précision d'un point mort à 90 %. Or l'étape 4 démontre que `p` n'est pas calibré et l'interdit dans toute porte. `p_ancré = Φ(z)` est déclaré « score borné, PAS une probabilité calibrée » dans la section 6 elle-même.

5. **L'argument de non-directionnalité survit formellement mais pas économiquement.** La symétrie miroir tient ($|m^*|$, z , σ_T , δ identiques sous mouvement miroir, prix complémentaire garanti par l'invariant). Mais une position miroir de taille miroir sur un pari à point mort 90 % reste un pari : la symétrie garantit l'absence de biais de côté, pas l'absence d'exposition au sens.

Les deux issues possibles, et il faut choisir avant de coder :

Option	Règle de prix	Ce que devient le mécanisme	Ce qu'il faut mesurer
A. Vraie tenue de marché	<code>prix_limite = min(arrondi_tick_bas(p_ancré - δ), bid_côté)</code> ou <code>bid + 1 tick</code>	Quote passive, économie = δ - frais, PnL de l'ordre de +0,075 \$ par fill, cohérente avec tout le raisonnement de l'étape 4	Le taux de fill réel (M3). L'écart e devient une condition d'éligibilité, plus un prix
B. Achat de valeur assumé	<code>prix_limite = ask_côté</code> , ordre taker explicite	Achat d'un binaire sous-évalué, détenu jusqu'au règlement. Point mort $\approx$ ask. Exige un q calibré	Le Brier de $\Phi(z)$ par sous-groupe sur ≥ 50 sessions, avant toute exécution

Mon avis : **option A**, et c'est net. Toute la démonstration de l'étape 4 (la décote rémunère σ_T , la contrepartie ne re-cote pas sur un événement silencieux, la dégressivité est arithmétique) décrit une activité de tenue de marché. L'option B contredit le résultat le mieux établi du corpus, à savoir qu'il n'existe aucun prédicteur (R^2 max 0,207) et que le carnet réplique m^* à $R^2 = 0,904$. Sous l'option B, le bot parie que $\Phi(z)$ bat le carnet, ce que le Brier par sous-groupe réfute déjà.

Et cette décision **détermine si l'étape 5 est dans le périmètre** : sous A, la sortie unique au règlement est cohérente et V1/V2/V5 sont hors sujet ; sous B, une position directionnelle remplie a besoin de stops, et le moteur de sortie de l'étape 5 devient obligatoire.

Faits, déductions, hypothèses : la ligne de partage

Statut	Contenu
Faits confirmés	Les 9 lignes du tableau « informations fiables » de la section 1, chacune avec un comptage explicite et une universalité déclarée. Plus : les 8 caractères d'encodage reconstitués.
Déductions (miennes, vérifiables)	R36 (la porte produit un ordre agressif), R37 (δ

	retenu sous son propre plancher), R38 (le coupe-circuit exclut la phase la plus large), R40 (deux formules de N), R41 (trois modèles de frais sans règle d'affectation), R46 (V1 exige une grandeur interdite).
Hypothèses	Le tick vaut 0,01 (inféré, pas de schéma). Φ est la normale standard. τ est le temps de fenêtre officiel. La plateforme accepte des ordres limites post-only.
Décisions proposées	Option A pour la règle de prix. Estimateur A unique pour σ_{30} . Étape 4 prioritaire sur l'étape 5 partout où elles divergent. M3 avant M1. Journalisation en rang 1.
À valider par vous	Les 12 points de la page 7.

1. Registre de correction (50 entrées)

Aucune correction n'est considérée comme intégrée tant qu'elle n'est pas reliée à un module, à une règle précise et à un test vérifiable. Les colonnes **Module** et **Test** renvoient aux pages 3 et 6.

Statuts : `confirmé` = comptage explicite dans les sources ou démonstration arithmétique ; `probable` = cohérent mais non compté ; `ambigu` = sources incompatibles sans arbitrage possible ; `à valider` = exige votre décision.

A. Bloquants de données (fail-closed permanent, 11 champs absents de 41/41 sessions)

ID	Source	Problème	Impact stratégique	Correction attendue	Module	Test	Statut
R01	E4 §0.7, §3, §8.6	<code>K</code> et <code>k_source</code> absents 0/41. Tous les agents ont utilisé un proxy ; écarts inféré contre ajusté de +0,25 / +12,25 / -4,50 \$	Un écart de 12,25 \$ inverse le règlement d'une session. Sans K, aucune affectation à un sous-groupe n'est fiable, donc aucun PnL par sous-groupe n'est interprétable	Lecture 1× à τ_0 via source officielle, <code>k_source == "officiel"</code> obligatoire, aucun proxy, aucune tolérance. Refus de la session entière sinon	<code>Anchor</code>	T-A1, T-A2	confirmé
R02	E4 §0.1, §3	<code>endDate</code> absent 0/41. <code>t_ms</code> est un relatif de capture, aucune date absolue	Un τ faux invalide simultanément σ_T , M, λ , P_max et la fenêtre. Une origine relative prise pour absolue déplace la fenêtre de règlement de 126 s au-delà d'un marché clos	<code>$\tau_0 = endDate - 300\text{ s}$</code> , lu 1×, jamais <code>time()</code> . Bloquant. Deux horloges séparées et jamais mélangées : τ -marché et capture	<code>Chronos</code>	T-C1, T-C2	confirmé
R03	E4 §0.1, §5.21	Profondeur L2 absente 0/41 : aucun niveau 2, aucune quantité, aucun prix par niveau, aucune annulation	T21 fail-closed par construction, donc le bot ne cote jamais . Capacité et dérapage réels non déterminables	<code>profondeur_cumulée(jusqu'au prix limite) ≥ floor(N)</code> . Fail-closed permanent tant que le champ manque. Ne jamais simuler une profondeur	<code>Prism</code> , <code>Risk</code>	T-P4	confirmé
R04	E4 §0.7, §5.22 ; E1 1.4 trou n°4	<code>position</code> et <code>n_ordres_en_vol</code> non réconciliés 0/41	T22 fail-closed. « Une position fantôme non réconciliée vaut	État local + réconciliation API obligatoire avant chaque décision.	<code>Positions</code>	T-PO1, T-PO2	confirmé

			refus ». Risque de double ordre	Inventaire vide réconcilié exigé			
R05	E4 §0.7, §5.5	Drapeau <code>HALT</code> absent 0/41 : rend la moitié de T5 invérifiable	Cotation possible pendant une suspension de marché	<code>HALT</code> absent ⇒ refus, code <code>BASIS_CORROMPU</code> / <code>HALT</code> . Fail-closed	<code>Warden</code>	T-W3	confirmé
R06	E4 §6 ; E1 O11	<code>fee_bps</code> réel absent 0/41	La garde de pré-vol plafonne à 130 bps à ask = 0,65 : elle est active, donc son absence est structurante	Garde conservée : $\text{fee_bps}/10^4 \cdot \min(\text{ask}; 1-\text{ask}) \leq 0,02 \cdot \text{ask} \cdot (1-\text{ask})$, sinon <code>HALT</code> . Lecture pré-vol + à chaque rotation	<code>Fees</code>	T-F3	confirmé
R07	E4 §0.6, §8.6	Les 8 horodatages de latence absents : <code>t_reception_oracle</code> , <code>t_reception_bbo</code> , <code>t_decision</code> , <code>t_envoi</code> , <code>t_ack</code> , <code>t_fill</code> , <code>t_annul_envoi</code> , <code>t_annul_ack</code>	Budget total non déterminable, donc aucun mécanisme ne peut être déclaré atteignable . La fenêtre utile est de 2 s et la traversée de seuil dure 0,300 s	Journalisation obligatoire, horloge serveur commune, précision ≤ 10 ms. Rang 1 d'implémentation	<code>Chronicle</code> , <code>Executor</code>	T-CH2, T-E5	confirmé
R08	E4 §0.1, §8.6	<code>order_id</code> , prix et quantité exécutés, <code>maker/taker</code> , codes de rejet : absents. Les changements de BBO sont visibles, jamais leur cause	Le taux de remplissage réel est inconnu. La table de décote est une table de contacts BBO , pas de fills. Aucun PnL au niveau du fill	M3 (instrumentation) obligatoire avant M1 en taille réelle. Reconstruction de la table de décote avec des fills	<code>Executor</code> , <code>Harness</code>	T-E1, T-H4	confirmé
R09	E4 §0.5, §8.6	<code>issue_officielle</code> et TWAP [270;300] complet absents. Le corpus n'a que des TWAP partiels [270;298] ; le point [299;300] manque systématiquement	Le test de non-directionnalité sous forme opérationnelle (PnL positif dans chacun des 4 sous-groupes) est inexécutable	Récupération de l'issue officielle par session. Sous-groupe affecté à chaque décision journalisée	<code>Chronicle</code> , <code>Harness</code>	T-H5	confirmé
R10	E4 §0.1, §8.6	<code>oracle.ts_src</code> vaut littéralement la chaîne <code>payload</code> sur 269/269 lignes	Aucune heure source, donc la latence causale oracle vers carnet n'est pas mesurable à une échelle sous-seconde. Le champ est inutilisable	Exiger un <code>ts_src</code> numérique , plus l'heure de réception plateforme et l'heure de capture locale, séparées	<code>Intake</code>	T-I5	confirmé

R11	E4 §0.3, §3, §8.5	<code>trades.direction</code> est un code ± 1 sans dictionnaire . Ni prix, ni côté contractuel, ni maker/taker	Le marqueur de flux Φ_{30} (seuil 800 \$) est inutilisable. Le côté agresseur n'est pas restructurable, donc aucune population ne peut être nommée	Φ_{30} et <code>FCO_FLUX_VETO_USD</code> retirés . Champ <code>side</code> + dictionnaire à obtenir. Réexamen ultérieur	— (retiré)	T-W7	confirmé
R12	E4 §0.5	Le carnet cesse de publier vers $\tau = 273$ s ; dernière valeur oracle à 298 s	Toute logique après 270 s travaille sur des données mortes	<code>$\tau_{\max} = 270$ s</code> . Annulation de tout ordre en vol à 270 s, aucune ouverture au-delà	<code>Chronos</code> , <code>Executor</code>	T-C4	confirmé

B. Réfutations et recalibrages issus des étapes 1 à 4

ID	Source	Problème	Impact stratégique	Correction attendue	Module	Test	Statut
R13	E4 §0.2, §5.14, remarques 2/4/31	La latence <code>lag_signé ∈ [+5;+60] s</code> postulée n'existe pas : $\text{argmax}_{\text{médian}} -0,75 \text{ s}$, 0/27 dans la bande, 26/27 avec $L \leq 0$. T14 refuse 721/721 et 543/543. Toutes les configurations du top 12 ont <code>use_T14 = False</code>	C'est la justification écrite de toute la stratégie qui tombe. T14 est un veto absolu qui explique à lui seul le PnL nul	T14 retiré de la porte, pas recalibré (élargir une bande jusqu'à ce qu'elle contienne la mesure est une tautologie). Journalisé comme indicateur de santé, alerte non bloquante si la médiane glissante 10 sessions dépasse +5 s	<code>Chronicle</code>	T-CH5	confirmé
R14	E4 remarques 5/6, §6	<code>$\sigma_{30} = 26,50$ \$</code> gelé en constante : erreur d'unité, facteur $\times 3,68$. Produit 0 entrée sur la totalité du corpus	Stérilise le système entier via σ_T , λ , M et z	<code>σ_{30}</code> mesuré en ligne, jamais gelé . Bornes de plausibilité [0,5 ; 40] \$. Refus si la fenêtre de 30 s n'est pas pleine	<code>Oracle</code>	T-O1, T-O2	confirmé
R15	E4 §2 (remarques 5/6) ; E5 §1.3	Deux estimateurs de σ_{30} coexistaient sans arbitrage : A = écart-type des incréments 1 s $\times \sqrt{30}$ (médiane 7,20 \$, n=21) ; B = écart-type des niveaux sur 30 s (médiane 2,25 \$, n=8). Facteur ~ 3	B sous-estime le risque d'un facteur 3. L'étape 5 suspend V3 pour cette raison	Estimateur A imposé , définition unique, identique entre entrée et sortie . Une seule fonction dans tout le code, sans variante	<code>Oracle</code>	T-O3	confirmé

R16	E4 remarque 1, §4.2	La formule à 22 tests est un filtre stérilisant : 0 entrée, conjonction complète 0/610	PnL 0,00 \$: satisfait le critère de non-directionnalité trivialement, échoue au critère « zéro événement capté »	Ne pas coder 22 conditions comme 22 informations. Six dimensions indépendantes : validité de mesure, temps restant, détermination z, écart carnet/ancrage, qualité de carnet, état interne	Warden , Signal	T-W1, T-S1	confirmé
R17	E4 remarque 18, §5.15, §6	T15 exige un saut d'ask ≤ 0,01 sur 3 s alors que le p95 mesuré est 0,10 à 0,20 (p99 0,13 à 0,29, max 0,30). Le pas médian réel du carnet est 2 cents. Le test refuse le marché normal	Seuil 10 à 20× trop serré : capte 0 événement. Intersection T15 ∧ T16 = 0	Remplacer par q95(Δask sur 3 s) mesuré en ligne , plancher 0,02, plafond 0,30. Durée de confirmation de l'écart : 2 s conservée	Prism	T-P2	confirmé
R18	E4 §5.13, §6 ; E1 K23	ASK_MEDIAN_DELTA = 0,03 alors que la chute p95 mesurée est 0,08 à 0,15 (p99 0,13 à 0,29)	L'anti-couteau refuse la respiration normale du carnet	q95(chute d'ask sur 3 s) en ligne, plancher 0,03, plafond 0,30. Fenêtre 3 s conservée. Cadence 4 Hz minimum	Prism	T-P3	confirmé
R19	E4 remarque 17, §4.3, §5.17	D_max = 0,114 comme condition d'entrée est dépassé dans 371/610 snapshots (60,8 %), D médian +0,224. « T17 est en contradiction mathématique avec la formule de p : le garde-fou rejette le signal du modèle lui-même ». Plus z monte, plus p→1, plus D monte, plus T17 refuse : T17 refuse exactement les instants les plus déterminés	Le garde-fou ferme la porte au mécanisme réel en protégeant contre un danger inexistant (le carnet réplique l'oracle à R² = 0,904, il ne sait rien de plus)	D_max conservé en coupe-circuit seulement , jamais en condition d'entrée. Premier seuil à réviser après les 500 sessions	Warden	T-W5	confirmé
R20	E4 remarque 38, §5.18, §6 ; E1 1.5 §4.3	P_max(τ) = min(0,70 + 0,00075τ ; 0,92) refuse 198/198, 387/619, 245/480, 154/682	Exclusion croissante avec τ : ferme la fenêtre tardive, qui est	Les trois constantes supprimées . Remplacées par un test d'espérance nette symétrique : (1 - ask) > 0,02·ask·(1-ask) + 0,02	Warden , Fees	T-W6	confirmé

		selon session. Le plafond de 0,92 est inerte (atteint à 293,3 s). Aucune régression n'étaye les trois constantes	justement la fenêtre retenue				
R21	E4 remarque 33, §0.4, §6 ; E1 1.3.bis §3	T20 s'écrit <code>spread ≤ 0,04</code> sans borne basse. Plage observée -0,01 à 0,19 : deux spreads croisés passent à tort . Et la sentinelle <code>-1</code> pour « indisponible » crée un <i>fail-open</i> accidentel ($-1 \leq 0,04$ est vrai)	Cotation sur un carnet croisé ou sur des champs absents	$s \in [0 ; 0,04]$, bornes strictes des deux côtés . Disponibilité des champs en booléen séparé , fail-closed. Jamais de sentinelle numérique	Prism , Kernel	T-P1, T-K3	confirmé
R22	E4 remarques 14/15, §5.5, §8.5	T5 <code>spot > oracle</code> est vrai 1943/1943 fois : il n'a jamais rien filtré. Le basis est structurellement à +42,1 \$, soit 5,8 σ_{30} , positif sur 100 % des lignes appariées. Le miroir <code>spot < oracle</code> n'a aucune action miroir	Test unilatéral inerte, directionnel , qui masque un décalage 12 à 29× supérieur à M. Ce n'est pas une inefficience, c'est une différence de source (spot instantané Binance contre TWAP 30 s Chainlink)	T5 rejeté . Requalifié en contrôle de cohérence de source : <code> basis - médiane_glissante_60s(basis) ≤ 3 · $\sigma_{\text{session}}(\text{basis})$</code> , fail-closed. Jamais une entrée	Warden	T-W2	confirmé
R23	E4 §5.12, §6, §8.5 ; E1 1.5 §4.1	T12 impose <code>[m*(t) - m*(t-15)]/15 ≥ -0,40 \$/s</code> : borne unilatérale qui interdit la pente négative et autorise toute pente positive	Sélectionne une trajectoire, donc parie sur le sens	Garde symétrique : <code> Δm_{15} /15 ≤ 0,45 \$/s</code> . <code>PENTE_DIVISEUR</code> supprimé (l'arbitrage E1 1.5 en secondes devient sans objet)	Prism	T-P5	confirmé
R24	E4 remarque 30, §5.19, §6	T19 vérifie <code>Brier(p) ≤ Brier(mid)</code> . L'agrégat passe (0,4345 contre 0,5334) uniquement par compensation : modèle 0,679 contre mid 0,554	<code>p</code> n'est pas une probabilité. Le conserver en condition d'entrée revient à faire décider un score non calibré	<code>p</code> reste un score de diagnostic journalisé, hors de toute porte . Garde exigée dans chacun des 4 sous-groupes , sur ≥ 50 sessions étiquetées, coefficients de Platt versionnés	Oracle , Gate	T-G4	confirmé

		sous le strike, 0,019 contre 0,498 au-dessus					
R25	E4 remarques 29/39, \$5.16, \$8.5	$D = p - ask$ mesuré signé : les trois plus grosses sous-évaluations détectées sont toutes du mauvais côté. Le signe de D change de camp avec l'issue	Signal de valeur anti-corrélé au résultat sur une session	Remplacé par $e(\tau) =$ $ p_{\text{ancré}, \text{côté}} - ask_{\text{côté}} $ en valeur absolue , côté déduit par miroir du signe de m*	Garnet	T- GA2	confirmé
R26	E4 remarque 19, \$3, \$6	La fenêtre d'entrée [60;240] ne contient pas l'information : $\tau_{\text{lock}} = 262$ s, soit 22 s après sa fermeture ; 16 à 18 % du chemin de prix est posté dans les 20 % finaux. Mouvement résiduel de l'oracle : 11,62 \$ en [60;180[contre 1,50 \$ en [180;240[	« Le postulat cherche à décider avant que l'issue existe. » Avant 180 s, toute position est à 1,6 σ_{30} d'exposition au sens	Fenêtre d'action déplacée en [180 ; 270] . $w(\tau)$ et le TWAP de règlement redeviennent actifs dans [240;270]. C'est une condition de non-directionnalité, pas un réglage de performance	Chronos , Signal	T-C3, T-S2	confirmé
R27	E4 \$5.11, \$6	$X_{60} < 3$ trop permissif : $X_{60} = 0$ mesuré à 100 % de réussite (n = 335) contre 99,5 %. Et $X_{60} = 0$ est le marqueur d'« oracle décroché »	Laisse passer les sessions à oracle oscillant, où l'écart médian est négatif (-0,0611 et -0,0547)	Seuil durci à $X_{60} = 0$. Fenêtre 60 s inchangée ; l'anomalie de fenêtre incomplète disparaît avec $\tau_{\text{min}} = 180$ s	Oracle , Warden	T-O5	confirmé
R28	E4 \$5.10, \$6	$FCO_PERSIST_S = 10$ s trop long au regard de la cadence oracle de 1 s	Réduit le nombre d'événements éligibles dans une fenêtre de 90 s	5 s cumulées de même signe avec marge. Reset si le signe de m* change ou si $trou_oracle > 8$ s	Oracle	T-O4	confirmé
R29	E4 \$6 ; E1 1.5 \$3.3	$K_ECART_MAX_USD =$ 5,00 \$: une tolérance de strike égale à M(240) annule le signal minimal	Rend la condition de marge vide de sens	Supprimé. Aucune tolérance sur K : officiel ou refus de session	Anchor	T-A2	confirmé
R30	E4 \$6 ; E1 1.6 \$4.3	$CAPITAL_DEMI = 2,50$ \$ et $RATIO_PLEIN =$ 1,5 : branche	Code mort qui suggere une flexibilité inexistante	Supprimés. Taille unique $N =$ $\text{floor}(\dots)$, capital $C = 5,00$ \$ (conflit 5,00 / 2,00 \$ tranché en faveur du PDF)	Risk	T-R2	confirmé

		arithmétiquement vide (21 s sur 195)					
R31	E4 §0.2, §6, §8.7.2 ; E1 1.3.bis §4.1	Évaluation à 1 Hz alors que la cadence médiane du carnet est 0,013 à 0,015 s (rapport 77:1). À 1 Hz, 98 % des mises à jour sont invisibles. Le krach d'ask de 0,279 s et la traversée de seuil de 0,300 s sont manqués (70 % du temps pour la seconde)	Les gardes anti-couteau et anti-traversée sont aveugles à l'événement qu'elles doivent détecter	Déclenchement par l'union des événements oracle et carnet, 4 Hz minimum sur le carnet. Rendu obligatoire	Intake , Arbiter	T-I3	confirmé
R32	E4 §8.7.1, §8.7.2, §8.7.3	Trois fonctions structurelles absentes des 22 tests : contrôle de complémentarité, annulation sur traversée du strike, garde d'universalité de la volatilité	Sans la première, cotation sur données corrompues. Sans la seconde, M1 peut devenir directionnel malgré toutes ses gardes. Sans la troisième, un calibrage de régime calme est appliqué à un régime agité	INVARIANT_ROMPU (≤ 1 tick), TRAVERSEE_SEUIL (annulation inconditionnelle et prioritaire, puis $X_{60} := 1$ pendant 60 s), REGIME_NON_REPRESENTATIF (ratio $\in [0,4 ; 2,5]$)	Warden , Regime , Executor	T-W4, T-RG1, T-E4	confirmé
R33	E4 §6	GEL_MAX_CARNET absent du PDF : seul le trou oracle était borné	Un carnet gelé passait inaperçu (P90 mesuré 1,19 s, max 6,5 à 9,2 s)	Nouveau seuil : 3 s . Avec GEL_MAX_ORACLE = 8 s et STALENESS_MAX = 2,5 s conservés	Warden	T-W3	confirmé
R34	E4 remarques 7/8, §2, §6	k_λ = 0,5513 utilisé comme réglage . Le balayage de Brier donne un optimum de 8,00 en S1 et 0,15 en S2 (facteur 53) : paramètre non identifiable	Confusion entre une identité algébrique et un hyperparamètre. Le facteur 53 ne réfute pas l'identité : il démontre que p n'est pas calibré	k_λ = 0,5513 = $\sqrt{3/\pi}$ codé en identité de conversion λ = 0,5513 · σ_T , non configurable, non optimisable. Confirmé par ajustement direct sur le mid (0,54 / 0,51 / 0,58, RMSE 0,06 à 0,09)	Oracle	T-O6	confirmé
R35	E4 §8.7.3, réserve finale	Le corpus couvre une seule journée (18/08/2026), volatilité annualisée implicite 6,9 %	Un δ_guard de 2 ticks calibré sur 6,9 % de volatilité est très inférieur au mouvement adverse d'un	Garde σ_30_session / médiane_glissante_20_sessions(σ_30) ∈ [0,4 ; 2,5] . Hors bornes : aucune cotation, alerte, révision obligatoire de δ_guard avant reprise	Regime	T-RG1, T-RG2	confirmé

		(9,5 / 5,8 / 5,3 % par session) contre 40 à 60 % typiques pour BTC. σ_{30} varie d'un facteur 6,6 intra-session et 7,6 inter-session	régime à 50 %. Toute constante calibrée sur ce corpus est spécifique à ce calme				
R50	E4 §4.3	T10 et T11 mesurent la même chose : « un oracle oscillant échoue aux deux, un oracle décroché passe aux deux »	Double comptage, fausse impression de deux filtres indépendants	Conserver les deux mais les journaliser comme une seule dimension dans l'analyse. Ne pas les compter comme deux confirmations	Warden	T-W1	probable

C. Erreurs internes du livrable retenu (détectées ici, absentes des sources)

 Ces six entrées ne figurent dans aucun fichier. Elles sont le résultat de la confrontation arithmétique des sections 6, 7.3 et 8.4 bis de l'étape 4 entre elles. Elles bloquent le codage.

ID	Source	Problème	Impact stratégique	Correction attendue	Module	Test	Statut
R36	E4 §7.3 (porte + règle de prix)	La porte garantit un ordre agressif. $e > \delta_{min} \Rightarrow p_{ancré} - \delta > ask \Rightarrow$ le prix limite dépasse le meilleur ask $\Rightarrow$ exécution immédiate en taker. <code>poster_passif()</code> ne peut jamais être passif quand la porte s'ouvre	Le PnL annoncé $N \cdot (\delta - \text{frais}) = +0,075 \$$ est inatteignable : c'est une capture de spread par quote passive. Le PnL réel est $+N \cdot (1 - ask - \text{frais})$ ou $-N \cdot ask$, soit +1,18 \$ ou -4,80 \$ à ask = 0,80 et N = 6. Point mort $\approx ask$, soit 80 à 96 % de réussite directionnelle exigée. « Slippage attendu 0 » est faux. Le taux de contact BBO (88 % / 49 %) et la validité de 2 s sont hors sujet	Décision requise. Option A : <code>prix_limite = min(arrondi_tick_bas(p_ancré - δ), bid_côté)</code> , ordre post-only, <code>e</code> devient une condition d'éligibilité. Option B : taker assumé au ask, mais alors $\Phi(z)$ doit être calibré par sous-groupe avant tout ordre, ce que R24 interdit aujourd'hui	Signal , Executor , Risk	T-S4, T-E2	à valider
R37	E4 §6 contre §8.4 bis	La décote retenue viole son propre plancher. §8.4	À $\delta = 0,02$, l'espérance est négative sous le modèle de frais	Décision requise. Soit $\delta = 0,03$ avec un taux de fill à mesurer, soit reconnaître que le plancher pertinent	Fees , Signal	T-F1, T-S5	à valider

		bis retient $\delta = 0,02$ (2 ticks) en phases [180;240[et [240;270[. §6 déclare $\delta_{\min} = \text{frais_AR} + 0,01 = 0,0207$ à $0,0375$ et le qualifie de non négociable	de sizing, « quel que soit le sens du sous-jacent ». Le premier multiple de tick admissible est 0,03 , dont le taux de contact n'a jamais été mesuré (mesures à 0,02 et 0,05 seulement, avec 70,92 % contre 85,20 % : la chute est nette)	est $\text{frais_AR_pnl} + 1 \text{ tick}$ et non $\text{frais_AR_sizing} + 1 \text{ tick}$, ce qui donne $\delta_{\min} \approx 0,0132$ et rend $\delta = 0,02$ admissible. Les deux sont défendables, mais pas les deux ensemble			
R38	E4 §0.4 contre §7.3	Le coupe-circuit exclut la phase la plus large. La porte de §7.3 contient $e \leq 0,114$, alors que l'écart médian mesuré $ p_{\text{ancré}} - \text{mid} $ vaut 0,4642 pt en [240;270[et 0,0900 pt en [180;240[	C'est exactement l'erreur structurelle que §4.3 reproche à T17 , réintroduite dans la porte retenue : le coupe-circuit refuse précisément la phase terminale que le mécanisme cible. En [240;270[, la porte refuse tout ou presque tout	Décision requise. Soit $D_{\max}$ est relevé à un quantile mesuré en ligne (cohérent avec R17/R18/R20), soit il est évalué hors de la conjonction d'entrée comme pure alerte de journalisation, soit la phase terminale est abandonnée. Le conserver en dur dans la porte reproduit la stérilité de l'étape 3	Warden	T-W5	à valider
R39	E4 §8.1 point 14 contre §0.4	Deux mesures incompatibles du même écart. La régression $e(\tau) = 0,04998 + 0,003358 \cdot \tau$ ($R^2 = 0,9582$ sur $\tau \geq 180$ s) donne 0,654 pt à $\tau = 180$ s et 0,957 pt à 270 s, alors que les médianes par phase du même document valent 0,090 et 0,464 pt	Un facteur 7 d'écart. L'une des deux grandeurs n'est pas $ p_{\text{ancré}} - \text{ask} $, ou la régression est par session avec des coefficients non transposables. Toute règle qui s'appuie sur la pente de $e(\tau)$ est actuellement infondée	Ne rien coder sur cette régression. Utiliser uniquement les médianes par phase, et recalculer $e(\tau)$ en ligne. Trancher en revenant à ETAP4. MD brut	Garnet	T-GA3	ambigu
R40	E4 §6 contre §7.3	Deux formules de sizing. §6 : $N = C/(\text{ask} + \text{frais} + 0,01)$. Pseudocode §7.3	Écarts de taille de ± 1 part selon l'ask, ce qui peut faire basculer la contrainte $N \geq$	Une seule fonction de sizing dans tout le code. Je retiens la version §6 (celle du PDF, qui fonde $\text{ask} \leq$	Risk	T-R1	confirmé

		: <code>N =</code> <code>plancher(5.00 /</code> <code>(ask +</code> <code>frais_AR))</code> , sans le <code>+ 0,01</code>	<code>PARTS_MIN = 5</code> et donc l'éligibilité elle-même	<code>0,98926</code>), sauf avis contraire			
R41	E4 §6, remarque 40 ; E5 §8.3	Trois modèles de frais sans règle d'affectation. <code>frais_AR_sizing =</code> <code>0,07*ask(1-ask) +</code> <code>0,01</code> ; <code>frais_AR_pnl =</code> <code>0,02*ask(1-ask)</code> ; <code>f^sell</code> de l'étape 5. L'écart est « assumé comme marge de sécurité » mais jamais formalisé	L'étape 5 le dit franchement : pénaliser l'achat d'un aller-retour alors que la simulation conserve jusqu'au règlement crée un biais de comparaison. Et c'est la cause directe de R37	Un module de frais unique exposant trois fonctions nommées et documentées : <code>fee_sizing(ask)</code> , <code>fee_settlement(ask)</code> , <code>fee_exit(ask)</code> . Interdiction de réutiliser l'une pour l'usage d'une autre, vérifiée par test	<code>Fees</code>	T-F1, T- F2	confirmé

D. Contradictions entre l'étape 4 et l'étape 5

ID	Version Étape 4	Version Étape 5	Contradiction exacte	Impact	Version privilégiée et raison	Statut
R42	M1 §7.3 : « sortie unique, au règlement , pas de sortie intermédiaire, parce que la thèse est que le temps travaille pour la position » ; « sortir tôt reviendrait à racheter l'incertitude qu'il vient de vendre »	Moteur complet de sortie anticipée V1, V2, V5, avec états <code>SELL_SIGNAL</code> , <code>SELL_SUBMITTED</code> , <code>SELL_FILLED</code> , <code>SELL_PARTIAL</code>	Les deux documents ne décrivent pas le même bot. L'un interdit ce que l'autre construit	Détermine si la moitié du travail de l'étape 5 est dans le périmètre. Impossible de figer la machine à états sans trancher	Dépend de R36. Sous option A (tenue de marché), l'étape 4 a raison : la sortie unique au règlement est cohérente et V1/V2/V5 sont hors sujet. Sous option B (achat de valeur), l'étape 5 devient obligatoire : une position directionnelle remplie a besoin de stops	à valider
R43	Fenêtre d'action [180 ; 270] , τ_{lock} = 262 s, verrou terminal implicite à 270 s	§9 : entrées M1 « concentrées vers τ ≈ 122 s », fenêtre de protection [60;120[, sortie par valeur [120;240[,	L'étape 5 raisonne entièrement sur la fenêtre [60;240] antérieure au déplacement.	Toutes les fenêtres de l'étape 5 sont à redériver	Étape 4. τ_{lock} = 262 s est mesuré ; 240 s est hérité du PDF. Et trois familles de mesures indépendantes	confirmé

		verrou terminal V6 à 240 s	Son verrou à 240 s forcerait la conservation à travers les 22 s exactes où l'issue se décide et où l'écart est le plus large (0,4642 pt)		réclament [180;270]. Réserve : τ_{lock} n'est mesuré que sur une session	
R44	$STALENESS_MAX = 2,5$ s (7/7, max observé 4,0 s), $GEL_MAX_ORACLE = 8$ s, $GEL_MAX_CARNET = 3$ s	V0 : $STALE_ORACLE$ à $\Delta_{oracle} > 3$ s, $STALE_ORACLE_HARD$ à > 7 s	Trois seuils divergents sur la même grandeur : 2,5 / 3 / 7 / 8 s	Un seuil de fraîcheur trop laxiste laisse coter sur un ancrage mort, qui est la seule référence de prix du bot	Étape 4. Ses seuils sont calibrés (7/7 et 8/8) ; l'étape 5 reconnaît elle-même que les siens « sont des paramètres de sécurité, non des constantes empiriquement validées »	confirmé
R45	T20 corrigé : $s \in [0 ; 0,04]$, bornes strictes des deux côtés	V7 : $s_t \leq s_{max}$ avec $s_{max} = 0,02$, aucune borne basse	V7 reproduit exactement le défaut de R21 : les deux spreads croisés à -0,01 passent. Et 0,02 contre 0,04	Vente sur carnet croisé	Étape 4 pour la borne basse (obligatoire des deux côtés). Pour la borne haute, 0,02 en sortie et 0,04 en entrée est défendable : une sortie subit le spread, une entrée passive le gagne. À documenter comme choix, pas comme mesure	confirmé
R46	p non calibré, hors de toute porte ; $p_{ancré} = \Phi(z)$ explicitement « score borné, PAS une probabilité calibrée »	V1 : $b_t - q_t \geq f^{sell}_t + \varepsilon$ avec q_t = probabilité que la position détenue gagne ; V5 : $b_t - f^{sell} \geq q_t - L_{tail}$	La condition centrale de vente exige la grandeur exacte que l'étape 4 interdit . L'étape 5 le pressent (« seuil désactivé ») sans faire le lien	V1 et V5 ne sont pas implémentables aujourd'hui, quel que soit le résultat sur les frais et le remplissage	Étape 4. V1 est codée mais désactivée par drapeau , avec q journalisé comme score et jamais consommé. Activation conditionnée à $Brier(q) \leq Brier(mid)$ dans les 4 sous-groupes sur ≥ 50 sessions	confirmé
R47	Les trois constantes de P_{max} (0,70 / 0,00075 / 0,92)	V5 : « règle pratique candidate $b_t \geq 0,92$ »	L'étape 5 ressuscite une constante que l'étape 4 a	Faux sentiment de validation empirique sur un seuil arbitraire	Étape 4. 0,92 n'est pas une constante validée. Remplacer V5 par	confirmé

	sont supprimées : le plafond est inerte, atteint à 293,3 s		supprimée faute de justification. Et l'audit mesure <code>ask > 0,92</code> sur 68 instants , pas des bids remplis : PnL « +0,13 \$ » décrit des instants, pas des transactions		le test d'espérance nette de R20 appliqué au bid. L'étape 5 a raison sur un point : ne jamais confondre <code>ask > 0,92</code> et <code>bid ≥ 0,92</code>	
R48	<code>TRAVERSEE_SEUIL</code> §8.7.2 : annulation inconditionnelle et prioritaire d'un ordre en vol au premier tick de signe opposé, puis $X_{60} := 1$ pendant 60 s. Aucune règle pour une position déjà remplie	V2 : invalidation directionnelle persistante, <code>sign(m*) ≠ side(position)</code> pendant <code>h = 20 à 30 s</code>	Ce n'est pas une contradiction mais une lacune réelle de l'étape 4 que l'étape 5 comble. Reste à trancher <code>h</code> : premier tick (étape 4, pour les ordres) contre 20 à 30 s (étape 5, pour les positions)	Une position dont le côté ancré s'inverse est un pari sur le côté que l'ancrage vient de désigner comme perdant. C'est le piège mesuré dans le corpus (oracle à +4,64 \$ à 240 s, effondrement de 22 \$ en 15 s, issue DOWN)	Les deux, sur des objets différents. Annulation au premier tick pour un ordre en vol (coût nul). Pour une position remplie, la sortie coûte le spread : la persistance de l'étape 5 est justifiée, mais <code>h = 20 à 30 s</code> ne repose que sur une session et aucune session UP ne teste la symétrie	à valider

E. Documents manquants

ID	Manque	Ce que je ne peux pas faire sans	Statut
R49	Étapes 1, 2, 3 ; <code>ETAP4.MD</code> brut (9 988 lignes) ; PDF 17 pages ; méthode d'achat ; journaux de sessions ; erreurs de sessions ; fragments de code de l'ancien bot ; CSV	Vérifier les citations. Recalculer les agrégats. Trancher R39. Confirmer l'inventaire des 60 termes de l'étape 1 et les 24 champs de journalisation. Identifier les erreurs réelles de l'ancien code (aucun fragment reçu, donc aucune erreur de code n'est dans ce registre : tout ce qui y figure est une erreur de <i>spécification</i>)	à fournir



Bilan du registre : 50 entrées. 43 confirmées, 1 probable, 1 ambiguë, 5 à valider par vous. 12 bloquants de données dont **4 empêchent physiquement le bot de coter** (R01, R02, R03, R04). 6 erreurs internes du livrable retenu, dont **R36 remet en cause la nature même du mécanisme**.

2. Stratégie cible reconstruite

 **La thèse en une phrase.** Le bot ne prédit rien. Il vend de l'attente sur la seule grandeur du système connue à l'avance : la décroissance déterministe du mouvement que l'oracle peut encore faire avant le règlement. La contrepartie ne re-cote pas cette décroissance parce que c'est un événement silencieux : rien ne bouge dans le carnet, et c'est précisément cela qui est l'information.

Cette thèse est **conditionnée à la résolution de R36 en option A**. Sous option B, la stratégie cible change de nature et devient un achat de valeur directionnel : je le signale à chaque endroit concerné.

2.1 Données nécessaires

Flux	Champs	Cadence requise	Horloge	Disponible
Oracle	<code>price</code> , <code>t_ms</code> , <code>ts_src</code> numérique	1 Hz nominal, trou max 8 s	Capture oracle	Partiel : <code>ts_src</code> inutilisable (R10)
Carnet	<code>yes_bid</code> , <code>yes_ask</code> , <code>no_bid</code> , <code>no_ask</code> + booléen de disponibilité par champ	4 Hz minimum	Capture carnet	Oui
Carnet L2	12 niveaux (prix, quantité), annulations	4 Hz minimum, ≤ 100 ms	Capture carnet	Non (R03) : bloquant
Spot	<code>price</code> , <code>t_ms</code>	Événementiel, médiane 0,38 s	Capture spot	Oui, usage intégrité seulement
Officiel	<code>K</code> , <code>k_source</code> , <code>endDate</code> , tick, <code>HALT</code> , <code>fee_bps</code>	1× à l'ouverture, <code>HALT</code> événementiel	Marché absolue, ≤ 100 ms	Non (R01, R02, R05, R06) : bloquant
Moteur	<code>order_id</code> , <code>t_ack</code> , <code>t_fill</code> , prix et qté exécutés, <code>maker/taker</code> , codes de rejet	Événementiel	Serveur, ≤ 10 ms	Non (R08) : bloquant
État interne	<code>position</code> , <code>n_ordres_en_vol</code> ,	Continu	Locale + serveur	Non (R04) : bloquant

	réconciliation API			
Résolution	<code>issue_officielle</code> , TWAP [270;300] complet	1×	Marché	Non (R09)
Trades	<code>t_ms</code> , <code>usd</code> , <code>side</code> + dictionnaire	Événementiel	Capture	Partiel : <code>direction</code> sans dictionnaire (R11). Journalisé, jamais consommé

Non négociable : 11 champs sur 24 sont absents. Le bot doit être construit pour refuser de coter tant qu'ils manquent, et non pour les contourner.

2.2 Conditions d'observation du marché

Le bot est **déclenché par l'union des événements oracle et carnet**, jamais par une horloge fixe (R31). Trois régimes d'observation :

- $\tau \in [0 ; 30[$: rien de décisionnel n'est calculable. La fenêtre de 30 s d'oracle n'est pas pleine, donc σ_{30} n'existe pas, donc σ_T n'existe pas, donc **le bot n'a pas de prix de référence**. Il **refuse**, il n'attend pas. La distinction compte : un refus s'écrit dans le journal, une attente ne s'écrit pas.
- $\tau \in [30 ; 180[$: il **mesure et se tait**. σ_{30} , σ_T , m^* , z , $p_{\text{ancré}}$ à chaque tick d'oracle ; e , spread, médiane 3 s, quantiles de saut et de chute à 4 Hz minimum. Il journalise tout et ne cote pas. Raison arithmétique et non prudentielle : le mouvement résiduel médian de l'oracle y vaut **11,62 \$ = 1,6 σ_{30}** , et la décote qui protégerait contre ce mouvement vaut **0,677 pt**, six fois le coupe-circuit.
- $\tau \in [180 ; 270]$: fenêtre d'action. Trois choses changent simultanément à 180 s : le mouvement résiduel tombe à **1,50 \$ (0,21 σ_{30})**, δ_{guard} tombe de 0,677 à 0,0157 pt (facteur 43), et l'écart carnet/ancrage, lui, **ne tombe pas**.

2.3 Grandeurs calculées et leur unique définition

τ	= horloge_marché() - (endDate - 300)	# jamais time(), jamais t_ms relatif
σ_{30}	= écart_type(incréments_1s(oracle,] τ -30 ; τ])) $\times \sqrt{30}$	# estimateur A, UNIQUE
$\sigma_T(\tau)$	= $\sigma_{30} \cdot \sqrt{((300 - \tau) / 30)}$	
$\lambda(\tau)$	= 0,5513 $\cdot \sigma_T(\tau)$	# identité $\sqrt{3/\pi}$, non configurable
$m^*(\tau)$	= oracle_last(τ) - K	
$z(\tau)$	= $ m^*(\tau) / \sigma_T(\tau)$	
côté	= OUI si $m^*(\tau) \geq 0$ sinon NON	# transformation miroir, PAS un pari
$p_{\text{ancré}}$	= $\Phi(z(\tau))$	# score borné, PAS une probabilité
$e(\tau)$	= $ p_{\text{ancré}} - \text{ask_côté}(\tau) $	# valeur absolue, jamais signée
$X_{60}(\tau)$	= nb_changements_de_signe(m^* ,] τ -60 ; τ])	
$P(\tau)$	= durée_cumulée_même_signe_avec_marge(τ)	
$v(\tau)$	= $ m^*(\tau) - m^*(\tau-15) / 15$	
$w(\tau)$	= min(1 ; max(0 ; (270 - τ) / 30))	# actif dans [240;270]
N	= floor(C / (ask + fee_sizing(ask) + 0,01))	# C = 5,00 \$; voir R40

À titre d'échelle avec $\sigma_{30} = 7,20$ \$: $\sigma_T(60) = 20,36$ \$, $\sigma_T(180) = 14,40$ \$, $\sigma_T(240) = 10,18$ \$, $\sigma_T(270) = 7,20$ \$, $\sigma_T(290) = 4,16$ \$. Division exacte par $\sqrt{5} = 2,236$ entre $\tau = 0$ et $\tau = 240$.

2.4 Conditions de validité (blocs, fail-closed, un code atomique chacun)

Bloc	Condition	Code de refus	Portée
Session	<code>endDate</code> présent et source officielle	<code>T0_APPROXIME</code>	Session entière refusée
	<code>K</code> présent et <code>k_source == "officiel"</code> , aucune tolérance	<code>K_NON_OFFICIEL</code>	Session entière refusée
Régime	<code>$\sigma_{30_session} / \text{médiane_20_sessions} \in [0,4 ; 2,5]$</code>	<code>REGIME_NON_REPRESENTATIF</code>	Session entière, alerte, révision de δ_{guard}
Fraîcheur	<code>âge_oracle ≤ 2,5 s</code>	<code>TICK_PERIME</code>	Instant
	<code>trou_oracle ≤ 8 s</code>	<code>TICK_TROU</code>	Instant, gel du compteur de persistance
	<code>trou_carnet ≤ 3 s</code>	<code>TICK_TROU</code>	Instant
	4 champs BBO présents (booléen, jamais sentinelle)	<code>SPREAD_INDISPO</code>	Instant
	<code>HALT</code> présent et faux	<code>HALT</code>	Instant
Intégrité	<code>$\text{bid_OUI} + \text{ask_NON} - 1 \leq 1 \text{ tick}$ et $\text{ask_OUI} + \text{bid_NON} - 1 \leq 1 \text{ tick}$</code>	<code>INVARIANT_ROMPU</code>	Instant
	<code>$\text{basis} - \text{médiane_glissante_60s} \leq 3 \cdot \sigma_{\text{session}}(\text{basis})$</code>	<code>BASIS_CORROMPU</code>	Instant
	<code>$\sigma_{30} \in [0,5 ; 40]$ \$ et fenêtre 30 s pleine</code>	<code>SIGMA_INDISPO</code>	Instant
Qualité de carnet	<code>$\text{spread} \in [0 ; 0,04]$, bornes strictes des deux côtés</code>	<code>SPREAD</code>	Instant + annulation
	<code>$\text{ask} \geq \text{méd}_{3s}(\text{ask}) - \text{q95}(\text{chute_3s})$</code>	<code>ASK_KRACH</code>	Instant + annulation, retour exigé 2 s

	$\max(\Delta ask \text{ sur } 3 \text{ s}) \leq q95(\Delta ask _{3s})$	ASK_INSTABLE	Instant
État interne	Profondeur cumulée jusqu'au prix limite $\geq floor(N)$	PROFONDEUR	Fail-closed permanent (R03)
	$position == \emptyset$, $n_ordres_en_vol == 0$, réconciliation OK, $floor(N) \geq 5$	OCCUPE / PARTS_MIN	Fail-closed permanent (R04)

2.5 Conditions d'entrée

Conjonction stricte, **un code de refus atomique par test**, évaluée dans l'ordre (le premier faux écrit le journal et arrête l'évaluation) :

- Tous les blocs de validité de 2.4 sont vrais.
- $180 \leq \tau \leq 270 \rightarrow$ sinon **HORS_FENETRE**.
- $z(\tau) \geq z_{min} = 1,0$ (statut **PROVISOIRE**) $\rightarrow$ sinon **MARGE_INSUFFISANTE**. Lecture : il faut un écart-type complet du chemin restant pour renverser l'issue.
- $X_{60}(\tau) = 0 \rightarrow$ sinon **CYCLICITE**. Marqueur d'« oracle décroché ».
- $P(\tau) \geq 5 \text{ s}$ de même signe avec marge $\rightarrow$ sinon **PERSISTANCE**.
- $v(\tau) \leq 0,45 \text{ \$/s}$ (garde symétrique) $\rightarrow$ sinon **RETOURNEMENT**.
- $e(\tau) > \delta_{min}$ de façon continue depuis $\geq 2 \text{ s} \rightarrow$ sinon **PAS_D_EDGE** ou **ECART_NON_CONFIRME**.
- $(1 - ask) > fee_settlement(ask) + 0,02 \rightarrow$ sinon **PRIX_MAX**. Test d'espérance nette symétrique, remplace **P_max**.
- $e(\tau) \leq D_{max} \rightarrow$ sinon **DECOTE_ABERRANTE**. **Contesté : voir R38**. Ce test refuse la phase [240;270[dont l'écart médian est 0,4642 pt.

Ce que le bot ne fait jamais en entrée : consommer **p** de Platt, **D** signé, **lag_signé**, la pente signée de **m***, **momentum_spot_5s**, **Φ₃₀**, ou le signe du basis. Ces grandeurs sont journalisées, jamais lues par une condition.

Décote et prix

```

δ_min(τ)  = fee_sizing(ask) + 0,01                # voir R37 : définition contestée
δ_guard(τ) = +∞                si τ < 180
           0,0157             si 180 ≤ τ < 240
           0,0107             si 240 ≤ τ < 270
           +∞                 si τ ≥ 270
δ(τ)      = max(δ_min(τ), δ_guard(τ))
δ_max     = 0,114                # borne de sécurité provisoire

```

Forme retenue : **enveloppe monotone décroissante par paliers**, jamais fonction continue. Raison : la médiane de δ_{guard} en phase milieu varie d'un facteur **37** entre sessions ; un ajustement continu produirait une fausse précision. Les régressions à $R^2 = 1,000$ sur deux points sont rejetées comme surajustées.

Cohérence de pilotage : $\sigma_T(180)/\sigma_T(255) = 1,67$ et $\delta_{\text{guard}}(180)/\delta_{\text{guard}}(255) = 1,47$. Les deux ratios sont du même ordre, ce qui confirme que σ_T pilote bien δ . **La dégressivité est une conséquence arithmétique, pas une hypothèse.**

Règle de prix : à trancher (R36).

```
Option A (recommandée) : prix_limite = min(arrondi_tick_bas(p_ancré -  $\delta$ ), bid_côté)
                        ordre POST-ONLY obligatoire, rejet si croisé
Option B                  : prix_limite = ask_côté, taker assumé
                        interdite tant que  $\Phi(z)$  n'est pas calibré par sous-groupe
```

Taille : $\min(\text{floor}(N), \text{profondeur_disponible})$, plancher **PARTS_MIN = 5**. Validité : 2 s.

2.6 Conditions de modification d'un ordre

Aucune modification en place. **Annulation puis recotation**, pour garantir l'unicité d' **order_id** et empêcher tout état ambigu :

- $|z(\tau) - z(\tau_{\text{envoi}})| \cdot d\Phi/dz > \delta(\tau)$: le déplacement de l'ancrage dépasse la décote elle-même.
- Franchissement de palier à $\tau = 240$ s : **annulation et recotation systématiques**, même si z n'a pas bougé, parce que δ change.
- Saut d'ask $> q95_{\text{saut}}$.

2.7 Conditions d'annulation

Par priorité décroissante. La première est évaluée **avant tout autre traitement** :

1. **TRAVERSEE_SEUIL** : $\text{signe}(m*(\tau)) \neq \text{signe}(m*(\tau_{\text{envoi}}))$ avec un ordre en vol $\rightarrow$ annulation **immédiate et inconditionnelle**, puis $X_{60} := 1$ ce qui interdit toute recotation pendant 60 s (dans [180;270], cela met le plus souvent fin à la session, et c'est voulu : une traversée tardive est la signature d'une session non tradable). Détection à **4 Hz minimum** : la traversée dure 0,300 s et est manquée 70 % du temps à 1 Hz. Budget d'annulation cible **< 300 ms**.
2. Rupture d'intégrité (n'importe quel code de 2.4).
3. $X_{60} > 0$, **spread** hors bornes, **ASK_KRACH**, **RETOURNEMENT**.
4. Expiration de validité à 2 s.
5. $\tau > 270$ s : annulation de tout ce qui reste, aucune ouverture au-delà.

Délai maximal d'annulation cible : **≤ 500 ms**, à mesurer.

2.8 Conditions de sortie



C'est le point le plus contesté du livrable (R42). Les deux politiques ci-dessous sont mutuellement exclusives. Je code les deux derrière un drapeau unique et vous choisissez.

Politique 1 (étape 4, cohérente avec l'option A de R36) : sortie unique au règlement

Aucune sortie intermédiaire. Justification : la thèse est que le temps travaille pour la position ; sortir tôt revient à racheter l'incertitude que le bot vient de vendre. La friction aller-retour est de **0,044 à 0,047 par unité** au voisinage de ask = 0,50 à 0,71 (mesure étape 5), soit environ 0,5 à 0,6 \$ de mouvement oracle : toute sortie plus fine que cette bande morte détruit de la valeur.

Seules exceptions autorisées, toutes de sécurité et toutes journalisées : `SUSPEND_DATA` (fraîcheur perdue), `SUSPEND_LIQUIDITY` (bid absent ou remplissage non confirmable), et l'invalidation directionnelle persistante ci-dessous.

Politique 2 (étape 5, obligatoire sous l'option B de R36) : sorties anticipées

Ordre d'évaluation strict, aucune réorganisation permise :

1. Pas de position → `NO_POSITION`.
2. Données périmées ou strike non résolu → `SUSPEND_DATA`. Seuils de l'**étape 4** (2,5 s / 8 s / 3 s), pas ceux de l'étape 5 (R44).
3. `bid ≤ 0` ou `spread ∉ [0 ; s_max_sortie]` ou remplissage non confirmable → `SUSPEND_LIQUIDITY`. **Borne basse obligatoire** (R45).
4. `τ ≥ 300` → `SETTLE`.
5. `τ ≥ τ_lock` → `HOLD_TO_SETTLEMENT`, sorties discrétionnaires interdites. `τ_lock = 262 s` mesuré (étape 4), **pas 240 s** (étape 5, R43).
6. **V5** prise de bénéfice, testée en premier car indépendante de l'invalidation. Sur le **bid**, jamais l'ask. Seuil 0,92 **refusé** (R47), remplacé par `bid - fee_exit(bid) ≥ q - L_tail` avec `q` désactivé tant que non calibré.
7. **V2** invalidation persistante : `sign(m*) ≠ side(position)` pendant `h` secondes consécutives. `h` à trancher (R48).
8. **V1** valeur : `bid - fee_exit(bid) ≥ V_hold + ε`. **Désactivée** : exige `q` calibré (R46).
9. **V3** érosion de marge `|m*| < k_exit · σ_T` avec hystérésis (`k_exit` de sortie < seuil de réentrée). **Diagnostic seulement**, déclencheur suspendu.
10. **V4** explosion de volatilité (`σ_T > 2 ×` médiane locale). **Jamais déclencheur autonome** : allonge `h`, réduit la taille, ou autorise V2.
11. Sinon `HOLD`.

États de sortie, non réductibles

`SELL_FILLED` · `SELL_PARTIAL` · `HOLD` · `HOLD_TO_SETTLEMENT` · `SETTLE_WIN` · `SETTLE_LOSS` · `SUSPEND_DATA` · `SUSPEND_LIQUIDITY`
· `NO_POSITION` · `UNKNOWN`

Sept contradictions interdites, codées en assertions d'exécution

- `SELL_FILLED` et `SUSPEND_LIQUIDITY` au même timestamp.
- `SELL_FILLED` après `$\tau\_lock$` sans sortie de sécurité explicitement journalisée.
- Vendre une position qui n'existe plus.
- Déclarer une vente au bid en utilisant l'ask.
- Déclarer un gain de vente sans confirmation de remplissage.
- Utiliser `p` pour une position NO sans convertir en `$q = 1 - p$` .
- Utiliser un strike inféré et un strike ajusté dans le même rejeu.

2.9 Conditions empêchant toute prise de position

Condition	Durée du blocage
Un des 4 bloquants de données (K, endDate, L2, réconciliation)	Permanent tant que le champ manque
<code>REGIME_NON_REPRESENTATIF</code>	Session entière
<code>$\tau < 180$</code> ou <code>$\tau > 270$</code>	Hors fenêtre
Oracle oscillant : <code>$X_{60} > 0$</code>	60 s glissantes minimum
Après une traversée du strike	60 s, donc en pratique la fin de session
<code>$\text{floor}(N) < 5$</code> , donc <code>ask > 0,98926</code> environ	Instant
Garde de frais : <code>$\text{fee_bps}/10^4 \cdot \min(\text{ask}; 1 - \text{ask}) > 0,02 \cdot \text{ask}(1 - \text{ask})$</code>	HALT jusqu'à rotation
Rejet d'ordre : 1 seul réessai, puis suspension	10 s

Le bot est inactif la plupart du temps, et c'est un résultat, pas un échec. Sur les 25 fenêtres analysées, les sessions à oracle oscillant (écart médian négatif, jusqu'à 8 traversées du strike) sont des sessions où la bonne décision est de ne rien faire, et deux des trois sessions du premier rapport en font partie. Chaque inaction est journalisée avec le code atomique du test bloquant, parce que **la distribution des non-entrées est la donnée qui permettra de réviser les seuils.**

2.10 Comportement en cas de donnée absente, invalide, retardée, dupliquée ou désordonnée

Situation	Comportement	Justification
Oracle muet	<code>TICK_TROU</code> , annulation immédiate, gel du compteur de persistance	L'ancrage est la seule référence de prix du bot : sans oracle frais, il n'a pas de prix
Oracle périmé (âge > 2,5 s)	<code>TICK_PERIME</code> , annulation	Max observé 4,0 s dans le corpus, donc le cas se produit
Carnet vide au meilleur niveau	<code>SPREAD_INDISPO</code> , annulation	Champs manquants 0,10 à 0,20 % des lignes
Champ absent avec sentinelle numérique	Interdit. Booléen de disponibilité séparé, fail-closed	La sentinelle <code>-1</code> créait un <i>fail-open</i> : <code>-1 ≤ 0,04</code> est vrai (R21)
Carnet croisé (spread < 0)	<code>SPREAD</code> , refus. Jamais interprété comme une opportunité	Plage observée -0,01 à 0,19 ; 2 lignes croisées passaient à tort
Invariant de complémentarité violé > 1 tick	<code>INVARIANT_ROMPU</code> , annulation	Signale une donnée corrompue ou un côté balayé non répercuté
Basis hors bande à 3σ	<code>BASIS_CORROMPU</code> , suspension de la session	Signature d'une désynchronisation d'horloge ou d'un flux décroché
Donnée dupliquée (même <code>t_ms</code> et même charge)	Idempotence : ignorée, comptée, journalisée	Un doublon ne doit jamais faire avancer un compteur de persistance
Donnée reçue dans le mauvais ordre (<code>t_ms</code> régressif)	Rejetée sur les flux monotones (oracle) ; sur le carnet, snapshot ignoré et compteur d'anomalies incrémenté. Aucun recalcul rétroactif d'une décision déjà prise	Une fenêtre glissante nourrie hors ordre produit un σ_{30} faux et donc un prix faux
Données contradictoires entre flux (oracle et spot divergent)	Le basis est l'arbitre. Hors bande $\Rightarrow$ suspension. L'oracle n'est jamais corrigé par le spot	L'oracle est la source unique de résolution ; le spot n'a aucune autorité sur l'issue
Rejet d'ordre par le moteur	Journaliser le code, un seul réessai, puis suspension 10 s.	Évite les boucles de réémission

	Événement compté comme non atteignable	
Exécution partielle	Journaliser la quantité effective. Elle mesure directement la profondeur au meilleur niveau , qui est le champ manquant n°1	Transforme un incident en mesure
Position fantôme (état local ≠API)	Refus de toute nouvelle décision, alerte, réconciliation forcée	R04

2.11 Conditions de passage du dry run au live

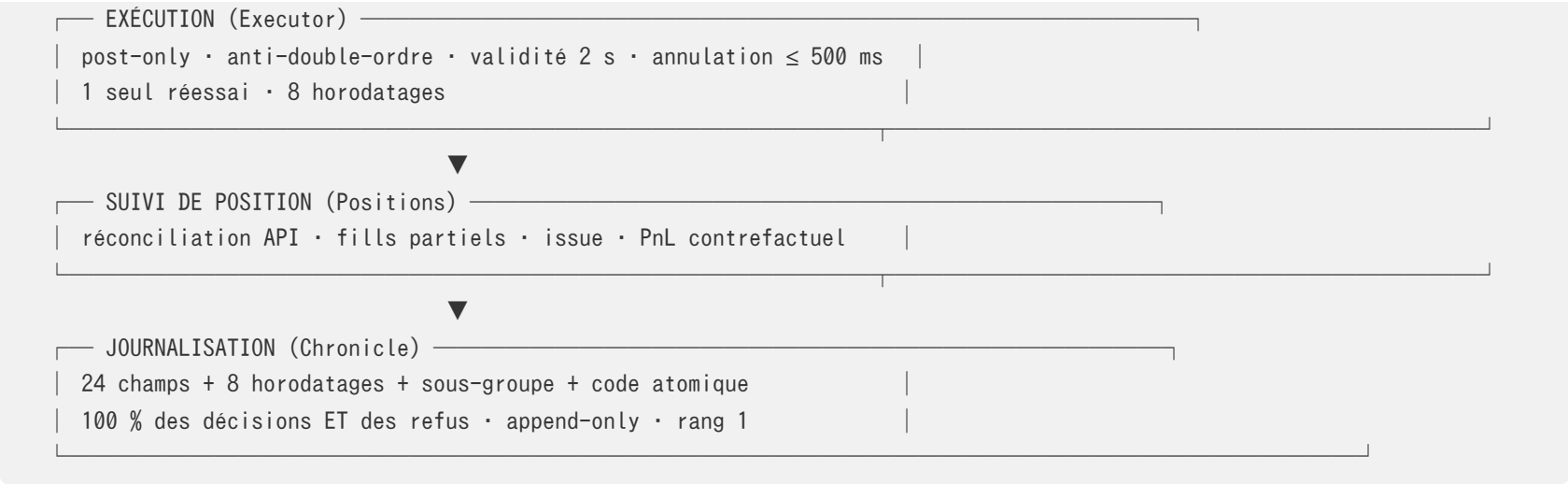
Cumulatives, dans cet ordre. Aucune n'est optionnelle :

1. Journalisation à **100 %** des décisions **et des refus**, avec code atomique unique attribuable, 24 champs + 8 horodatages + sous-groupe. Zéro ligne écrite après confirmation.
2. `endDate` , `K` , `k_source` , `HALT` , `fee_bps` , tick présents et officiels sur **≥ 99 %** des sessions. Écart `K_officiel - K_proxy` mesuré et publié. **Si `K` officiel est indisponible : arrêt du projet en l'état.**
3. Zéro cotation sur données incohérentes. Taux de refus par code cohérent avec le corpus (invariant rompu ≈ 0,2 %, spread hors bornes ≈ 14 %). PnL 0,00 \$ dans chaque sous-groupe.
4. σ_{30} mesuré en ligne, **zéro constante gelée**. Médiane sur 500 sessions publiée avec P10/P90. Sortie de `[0,5 ; 40]` sur ≤ 5 % des sessions.
5. Niveau 2 disponible sur ≥ 99 % des snapshots à 4 Hz. Réconciliation de position à 100 %, zéro position fantôme.
6. **M3 exécuté sur ≥ 100 sessions** à 5 parts, taux de fill réel mesuré par phase et par `$\delta \in \{0,01 ; 0,02 ; 0,03 ; 0,05 ; 0,08\}$` . Table de décote reconstruite avec des **fills**. PnL par sous-groupe renseigné pour la première fois. Budget de mesure : au plus **400 \$** d'exposition sur 100 sessions.
7. ≥ 0,5 événement éligible par session en moyenne.
8. Aucun test ne refuse **ni 0 % ni 100 %** des snapshots ; taux de refus par test entre 5 % et 30 %. **Un test qui refuse 100 % est faux par construction : c'est exactement l'erreur de T14 et T15.**
9. PnL net par événement > 0 après frais réels **dans chaque sous-groupe**.
10. Délai d'annulation mesuré ≤ 500 ms au P90. **Zéro fill sur un ordre dont le côté ancré a changé** (garde binaire : un seul suffit à invalider).
11. **PnL agrégé strictement positif dans CHACUN des quatre sous-groupes** (oracle en hausse / en baisse ; OUI gagne / NON gagne) sur ≥ 200 sessions, avec ≥ 100 sessions par sous-groupe. Ratio PnL / capital engagé > +1,0 % par événement rempli.

Critère de retrait, non négociable : un seul sous-groupe négatif entraîne le retrait immédiat du mécanisme. Un PnL positif obtenu avec un sous-groupe négatif est un pari, quel que soit l'agrégat. Et si aucune valeur de `z_min`, `delta_guard` ou `D_max` ne garde les quatre sous-groupes positifs, le mécanisme est **retiré, pas réajusté**.

2.12 Flux logique complet





Le journal n'est pas en bout de chaîne par commodité : il est le rang 1 d'implémentation. Aucune ligne de code de trading n'est écrite avant lui, et si le taux de journalisation descend sous 100 %, tout le reste est invalide.

3. Architecture complète (19 modules)

Principe directeur : **six dimensions indépendantes, pas 22 tests** (R16). Un module par responsabilité, aucune responsabilité partagée, aucun module qui à la fois mesure et décide.

Règle d'or : **la logique métier ne connaît ni le réseau, ni le disque, ni l'horloge système.** **Chronos** est la seule source de temps, **Intake** la seule source de données externes, **Executor** le seul émetteur d'ordres, **Chronicle** le seul écrivain. Tout le reste est pur et testable sans I/O.

3.1 Graphe de dépendances et ordre de construction

niveau 0	Kernel	————— (aucune)
niveau 1	Chronicle	← Kernel
niveau 2	Chronos	← Kernel, Chronicle
	Fees	← Kernel
niveau 3	Intake	← Kernel, Chronos, Chronicle
niveau 4	Anchor	← Kernel, Chronos, Intake
niveau 5	Oracle	← Kernel, Chronos, Intake, Anchor
	Garnet	← Kernel, Chronos, Intake, Fees, Oracle
niveau 6	Regime	← Oracle, Chronicle
	Prism	← Garnet, Oracle, Chronos
niveau 7	Warden	← Anchor, Oracle, Garnet, Prism, Regime, Positions
niveau 8	Positions	← Kernel, Chronicle (construit avant Warden, cycle brisé par une interface d'état en lecture)
	Risk	← Garnet, Fees, Positions, Kernel
niveau 9	Signal	← Warden, Oracle, Garnet, Risk, Chronos
	Exit	← Warden, Oracle, Garnet, Fees, Positions, Chronos
niveau 10	Arbiter	← Signal, Exit, Risk, Positions, Warden, Chronicle
niveau 11	Executor	← Arbiter, Positions, Chronicle, Kernel
niveau 12	Harness	← tous (rejeu, dry run)
niveau 13	Gate	← Harness, Chronicle

Le seul cycle potentiel est **Warden ↔ Positions** (Warden doit vérifier l'état interne, Positions doit être validé). Il est brisé en construisant **Positions** au niveau 8 et en n'exposant à **Warden** qu'une interface de **lecture seule** **PositionSnapshot**. Aucune autre dépendance circulaire n'existe.

**Ordre de construction imposé par les dépendances :** Kernel, Chronicle, Chronos, Fees, Intake, Anchor, Oracle, Garnet, Regime, Prism, Positions, Risk, Warden, Signal, Exit, Arbiter, Executor, Harness, Gate.

3.2 Spécification module par module

1. Kernel

1. **Nom** : `Kernel`
 2. **Objectif** : définir toutes les structures de données, unités et énumérations communes, une seule fois, de façon immuable.
 3. **Responsabilités** : types de snapshots ; énumération des 22 + 3 codes de refus atomiques ; énumération des 10 états de décision ; types d'unités distincts pour dollars, points de contrat, secondes de fenêtre et secondes de capture ; arrondi au tick ; **booléens de disponibilité explicites, jamais de sentinelle numérique**.
 4. **Données reçues** : aucune.
 5. **Données produites** : `OracleTick` , `BookSnapshot` , `SpotTick` , `TradeTick` , `SessionMeta` , `MeasureSet` , `ObservedTerms` , `Decision` , `OrderIntent` , `RefusalCode` , `Phase` , `Side` , `Subgroup` .
 6. **Fonctions publiques** : `round_down_to_tick(price, tick)` , `round_up_to_tick(...)` , `mirror_side(sign)` , `complement_price(price)` , `is_available(field)` , `Phase.of(tau)` , `Subgroup.of(oracle_direction, winning_side)` .
 7. **Dépendances** : aucune. C'est la racine.
 8. **Règles métier** : tick = 0,01 (**hypothèse inférée, à confirmer par l'API**) ; `complement_price(p) = 1 - p` par l'invariant de complémentarité ; les deux horloges sont des types **incompatibles**, toute addition entre elles est une erreur de compilation ou de type.
 9. **Cas d'erreur** : prix hors [0;1] ; tick nul ou négatif ; mélange d'horloges ; lecture d'un champ indisponible.
 10. **Journalisation** : aucune (module pur). Les erreurs de type lèvent.
 11. **Tests** : T-K1 arrondi au tick dans les deux sens, y compris sur les valeurs limites 0,00 et 1,00 ; T-K2 `complement_price` est une involution ; **T-K3 aucune sentinelle numérique n'existe dans le code (test statique sur la base de code, couvre R21)** ; T-K4 le mélange d'horloges échoue.
 12. **Critères de validation** : 100 % de couverture ; zéro valeur magique non nommée ; toutes les constantes résolues depuis `config` , aucune littérale dans le code métier.
-

2. `Chronicle`

1. **Nom** : `Chronicle` (journal de stratégie)
2. **Objectif** : garantir la traçabilité de **chaque** décision et de **chaque** refus. C'est le rang 1 d'implémentation : aucune ligne de code de trading avant lui.
3. **Responsabilités** : écriture append-only ; 24 champs + 8 horodatages + sous-groupe + code atomique ; sérialisation déterministe ; rotation par session ; indicateurs de santé hors ligne (`lag_signé` , taux de refus par code, distribution de `e( $\tau$ )`).
4. **Données reçues** : tout événement de décision, de refus, d'ordre, de fill, d'annulation, de rejet, d'alerte.
5. **Données produites** : fichier de journal par session ; agrégats de santé ; table de décote reconstruite depuis les fills.
6. **Fonctions publiques** : `log_decision(...)` , `log_refusal(code, context)` , `log_order_lifecycle(...)` , `log_session_open/close(...)` , `log_alert(...)` , `health_report(n_sessions)` .
7. **Dépendances** : `Kernel` .

8. **Règles métier** : un refus s'écrit, une attente ne s'écrit pas (le bot refuse, il n'attend jamais) ; un seul code atomique par refus, celui du premier test faux ; `momentum_spot_5s` , `pente_m_15s` , `$\Phi_{30}$` , `lag_signé` , `p` , `D` sont journalisés et leur consommation par une règle est interdite ; zéro ligne écrite après confirmation.
 9. **Cas d'erreur** : disque plein ; échec d'écriture ($\Rightarrow$ **arrêt du bot**, pas de dégradation silencieuse) ; horodatage manquant.
 10. **Journalisation** : lui-même, plus un compteur d'échec d'écriture.
 11. **Tests** : T-CH1 100 % des décisions et refus journalisés sur un rejeu complet ; **T-CH2 les 8 horodatages présents sur chaque ordre (R07)** ; T-CH3 unicité et atomicité du code de refus ; T-CH4 un échec d'écriture arrête le bot ; **T-CH5 `lag_signé` est journalisé et jamais lu par une condition (test statique, R13).**
 12. **Critères de validation** : taux de journalisation = **100 %** sur 500 sessions. En dessous, tout le reste est invalide.
-

3. Chronos

1. **Nom** : Chronos
 2. **Objectif** : être la **seule** source de temps du système et résoudre l'origine de fenêtre.
 3. **Responsabilités** : `$\tau_0 = \text{endDate} - 300 \text{ s}$` lu **1×** à l'ouverture ; calcul de `$\tau$` ; détermination de la phase ; séparation stricte des deux horloges (τ -marché et capture) ; `$w(\tau)$` .
 4. **Données reçues** : `endDate` officiel, horloge marché.
 5. **Données produites** : `$\tau$` , `Phase` , `$w(\tau)$` , `temps_restant` .
 6. **Fonctions publiques** : `open_session(end_date) -> Result` , `tau_now()` , `phase(tau)` , `w(tau)` , `is_in_action_window(tau)` , `is_locked(tau)` .
 7. **Dépendances** : `Kernel` , `Chronicle` .
 8. **Règles métier** : jamais `time()` comme origine, jamais `t_ms` relatif ; `endDate` absent $\Rightarrow$ `T0_APPROXIME` $\Rightarrow$ **session entière refusée** (R02) ; fenêtre d'action `[180 ; 270]` (R26) ; `$\tau_{\text{lock}} = 262 \text{ s}$` mesuré, **configurable et à confirmer** (n = 1 session) ; paliers 180 / 240 / 270.
 9. **Cas d'erreur** : `endDate` absent, non officiel, ou incohérent (τ négatif ou > 300) ; dérive d'horloge.
 10. **Journalisation** : ouverture ou refus de session ; τ_0 retenu ; chaque franchissement de palier.
 11. **Tests** : T-C1 refus de session sans `endDate` ; T-C2 le mélange des deux horloges est impossible ; T-C3 la fenêtre d'action est exactement `[180;270]` et couvre `$\tau_{\text{lock}} = 262 \text{ s}$` (R26) ; T-C4 annulation générale et fermeture à 270 s (R12) ; T-C5 `$w(\tau)$` vaut 1 à 240 s, 0 à 270 s, linéaire entre.
 12. **Critères de validation** : `endDate` officiel sur $\geq 99 \%$ des sessions ; zéro décision prise avec un τ approximé.
-

4. Anchor

1. **Nom** : Anchor
2. **Objectif** : résoudre et certifier le strike, et dériver le côté par transformation miroir.

3. **Responsabilités** : lecture 1× de `K` et `k_source` ; refus de session si non officiel ; calcul de `m* = oracle - K` ; `side = mirror(sign(m*))` .
4. **Données reçues** : `K` , `k_source` officiels ; dernier prix oracle.
5. **Données produites** : `K` certifié, `m*( $\tau$ )` , `side` , `sign_changed` (drapeau de traversée).
6. **Fonctions publiques** : `resolve_strike(payload) -> Result<Strike>` , `m_star(oracle_price)` , `anchored_side(m_star)` , `crossed_since(reference_sign)` .
7. **Dépendances** : `Kernel` , `Chronos` , `Intake` .
8. **Règles métier** : `k_source  $\neq$  "officiel"` $\Rightarrow$ `K_NON_OFFICIEL` $\Rightarrow$ **session entière refusée ; aucun proxy, aucune tolérance** (`K_ECART_MAX_USD` supprimé, R29) ; le signe de `m*` sert **uniquement** à choisir le côté, **jamais comme condition** (R25, remarque 28 de l'étape 4).
9. **Cas d'erreur** : `K` absent, proxy détecté, `K` hors bornes de plausibilité par rapport au spot.
10. **Journalisation** : `K` , `k_source` , `écart K_officiel - K_proxy` si les deux sont disponibles (exigé au rang 2), chaque traversée de signe avec son τ .
11. **Tests** : T-A1 **aucun proxy n'est jamais utilisé (test statique + rejeu, R01)** ; T-A2 refus de session sans `k_source` officiel et absence de toute tolérance (R29) ; T-A3 le signe de `m*` n'apparaît dans aucune condition d'entrée (test statique) ; T-A4 symétrie parfaite du choix de côté sous jeu de données miroir.
12. **Critères de validation** : `K` officiel sur ≥ 99 % des sessions. Si `K` officiel est indisponible, l'étape 4 conclut à l'arrêt du projet en l'état.

5. `Oracle`

1. **Nom** : `Oracle` (mesure des données brutes, TWAP/FIX)
2. **Objectif** : produire toutes les grandeurs dérivées de la série oracle, avec une définition unique et mesurée en ligne.
3. **Responsabilités** : `$\sigma_{30}$` (**estimateur A exclusivement**), `$\sigma_T$` , `$\lambda$` , `$z$` , `p_ancré =  $\Phi(z)$` , `$X_{60}$` , `P` , `v` , TWAP 30 s, mouvement résiduel.
4. **Données reçues** : série de `OracleTick` validée, `$\tau$` , `K` .
5. **Données produites** : `MeasureSet` = { `$\sigma_{30}$` , `$\sigma_T$` , `$\lambda$` , `$z$` , `p_ancré`, `$X_{60}$` , `P`, `v`, `twap30`, `age`, `gap`}.
6. **Fonctions publiques** : `sigma_30(window)` , `sigma_T(sigma_30, tau)` , `lambda_of(sigma_T)` , `z_of(m_star, sigma_T)` , `anchored_price(z)` , `x60(window)` , `persistence(window)` , `velocity(window_15s)` , `twap_30(window)` .
7. **Fonctions internes** : `_increments_1s(series)` , `_rescale_to_30s(sd)` , `_sign_changes(series)` .
8. **Dépendances** : `Kernel` , `Chronos` , `Intake` , `Anchor` .
9. **Règles métier** : `$\sigma_{30}$` = **écart-type des incréments à 1 s $\times \sqrt{30}$, une seule implémentation, identique entre entrée et sortie** (R15) ; jamais gelée (R14) ; refus si fenêtre 30 s incomplète ou `$\sigma_{30} \notin [0,5 ; 40]$  $` ; `$\lambda = 0,5513 \cdot \sigma_T$` en **identité non configurable** (R34) ; `p_ancré =  $\Phi(z)$` est un **score borné, jamais une probabilité** ; `p` de Platt calculé mais **hors de toute porte** (R24) ; `$X_{60} = 0$` exigé (R27) ; `P  $\geq 5$  s` (R28), reset si le signe change ou si `trou_oracle > 8 s` .

10. **Cas d'erreur** : fenêtre incomplète ; σ_{30} hors bornes ; division par zéro à $\tau \rightarrow 300$ ($\sigma_T \rightarrow 0$) ; trou d'oracle.
11. **Journalisation** : le `MeasureSet` complet à chaque évaluation ; `p` et `D` comme diagnostics ; volatilité annualisée implicite par session.
12. **Tests** : **T-O1** σ_{30} n'est jamais lue depuis une constante (test statique, R14) ; T-O2 refus si fenêtre incomplète ou hors bornes ; **T-O3 une seule implémentation de σ_{30} existe dans la base de code (R15)** ; T-O4 reset de persistance sur changement de signe et sur trou (R28) ; T-O5 $X_{60} = 0$ refuse une session à 8 traversées (R27) ; T-O6 k_λ n'est pas configurable (R34) ; T-O7 σ_T décroît en $\sqrt{\cdot}$ et vaut 14,40 / 10,18 / 7,20 \$ à 180 / 240 / 270 s pour $\sigma_{30} = 7,20$ \$; T-O8 σ_T bornée inférieurement pour éviter la division par zéro.
13. **Critères de validation** : médiane de σ_{30} sur 500 sessions publiée avec P10/P90 ; sortie de `[0,5 ; 40]` sur ≤ 5 % des sessions, sinon l'estimateur est faux.

6. `Garnet`

1. **Nom** : `Garnet` (termes observés du carnet)
2. **Objectif** : produire toutes les grandeurs dérivées du carnet, en valeur absolue et sans jamais introduire de signe directionnel.
3. **Responsabilités** : `ask_côté` , `bid_côté` , `spread` , `mid` , $e(\tau) = |p_{\text{ancré}} - \text{ask_côté}|$, `med3s(ask)` , `q95(| $\Delta \text{ask}$ |_3s)` , `q95(chute_3s)` , `basis` , médiane glissante 60 s du basis, `$\sigma_{\text{session}}(\text{basis})$` , contrôle des deux invariants, `$\delta_{\text{min}}$` , `$\delta_{\text{guard}}$` , `$\delta$` .
4. **Données reçues** : `BookSnapshot` validé, `SpotTick` , `p_ancré` , τ , `Phase` .
5. **Données produites** : `ObservedTerms` = {ask, bid, spread, mid, e, med3s, q95_jump, q95_drop, basis, basis_band, invariant_error, δ_{min} , δ_{guard} , δ }.
6. **Fonctions publiques** : `side_prices(book, side)` , `spread_of(book, side)` , `gap(anchored, ask)` , `rolling_median(series, window)` , `q95_jump(...)` , `q95_drop(...)` , `basis_of(spot, oracle)` , `invariant_error(book)` , `decote(tau, ask)` .
7. **Dépendances** : `Kernel` , `Chronos` , `Intake` , `Fees` , `Oracle` .
8. **Règles métier** : `e` toujours en valeur absolue (R25) ; le prix du côté NON est **dérivé du côté OUI** par l'invariant, ce qui divise par deux la surface d'erreur ; les quantiles sont **mesurés en ligne**, jamais constants (R17, R18), avec plancher et plafond ; $\delta = \max(\delta_{\text{min}}, \delta_{\text{guard}})$, enveloppe monotone **par paliers**, jamais ajustement continu ; le basis sert **exclusivement** au contrôle d'intégrité (R22) et n'entre dans aucune régression prédictive.
9. **Cas d'erreur** : champ BBO absent ; spread négatif ; invariant violé ; fenêtre de quantile incomplète ; $\delta_{\text{min}} > \delta_{\text{max}}$.
10. **Journalisation** : `ObservedTerms` complet ; distribution de $e(\tau)$ par phase ; fréquence de dépassement de `D_max` .
11. **Tests** : T-GA1 le prix NON dérivé égale le prix NON observé à 1 tick près sur 1 025 lignes ; **T-GA2 `e` est toujours ≥ 0 et le signe de `D` n'apparaît nulle part (R25)** ; **T-GA3 aucune règle ne consomme la**

régression $e(\tau) = 0,04998 + 0,003358 \tau$ (R39) ; T-GA4 les quantiles ne sont jamais des constantes (test statique) ; T-GA5 δ est monotone décroissante par paliers et $\delta \geq \delta_{\min}$ toujours (couvre R37) ; T-GA6 le basis n'alimente aucun signal (test statique).

12. **Critères de validation** : taux de refus par test de microstructure entre 5 % et 30 % ; **aucun test ne refuse ni 0 % ni 100 %**.

7. Fees

1. **Nom** : Fees
2. **Objectif** : centraliser les trois modèles de frais et **empêcher toute réutilisation croisée** (R41).
3. **Responsabilités** : fee_sizing, fee_settlement, fee_exit ; garde de pré-vol sur fee_bps ; bande morte aller-retour.
4. **Données reçues** : ask, bid, fee_bps de l'API.
5. **Données produites** : coûts par unité, bande morte, drapeau de garde de frais.
6. **Fonctions publiques** : $\text{fee_sizing}(\text{ask}) = 0,07 \cdot \text{ask} \cdot (1 - \text{ask}) + 0,01$; $\text{fee_settlement}(\text{ask}) = 0,02 \cdot \text{ask} \cdot (1 - \text{ask})$; $\text{fee_exit}(\text{bid}) \leftarrow$ à définir, paramètre de configuration, jamais un alias de fee_sizing ; $\text{fee_guard_ok}(\text{fee_bps}, \text{ask})$; $\text{dead_band}(\text{ask}, \text{spread})$.
7. **Dépendances** : Kernel.
8. **Règles métier : interdiction stricte** d'appeler fee_sizing dans un calcul de PnL ou de sortie, et réciproquement ; garde $\text{fee_bps} / 10^4 \cdot \min(\text{ask}, 1 - \text{ask}) \leq 0,02 \cdot \text{ask} \cdot (1 - \text{ask})$, sinon HALT (active : plafonne à 130 bps à ask = 0,65) ; bande morte mesurée 0,044 à 0,047 par unité au voisinage de ask = 0,50 à 0,71.
9. **Cas d'erreur** : fee_bps absent ($\Rightarrow$ fail-closed, R06) ; ask hors [0;1].
10. **Journalisation** : les trois frais à chaque décision, séparément nommés ; fee_bps à chaque rotation.
11. **Tests** : T-F1 fee_sizing n'est jamais appelée depuis un calcul de PnL ni de sortie (test statique, R41 et R37) ; T-F2 les trois fonctions sont distinctes et aucune n'est un alias ; T-F3 la garde de pré-vol refuse à fee_bps absent (R06) ; T-F4 la bande morte reproduit 0,044 à 0,047 aux asks du corpus.
12. **Critères de validation** : PnL net par événement > 0 après fee_settlement dans chaque sous-groupe.

8. Prism

1. **Nom** : Prism (filtres de microstructure)
2. **Objectif** : refuser les instants où le carnet n'est pas exploitable, sans jamais introduire de direction.
3. **Responsabilités** : bornes de spread des deux côtés ; anti-couteau ; stabilité d'ask ; garde de vitesse symétrique ; profondeur.
4. **Données reçues** : ObservedTerms, MeasureSet, profondeur L2.
5. **Données produites** : Verdict = {ok, code, valeur mesurée, seuil appliqué}.

6. **Fonctions publiques** : `spread_ok(spread)` , `no_knife(ask, med3s, q95_drop)` , `ask_stable(max_jump, q95_jump)` , `velocity_ok(v)` , `depth_ok(depth, n)` .
 7. **Dépendances** : `Garnet` , `Oracle` , `Chronos` .
 8. **Règles métier** : `spread  $\in [0 ; 0,04]$` , **bornes strictes des deux côtés** (R21, R45) ; seuils de saut et de chute = quantiles mesurés en ligne avec plancher et plafond (R17, R18) ; garde de vitesse **symétrique** | `$\Delta m^*_15 / 15 \leq 0,45$  $/s` (R23) ; profondeur **fail-closed permanent** (R03) ; évaluation à **4 Hz minimum** (R31).
 9. **Cas d'erreur** : quantile non calculable (fenêtre incomplète) $\Rightarrow$ refus ; profondeur absente $\Rightarrow$ refus ; `med3s` non calculable $\Rightarrow$ refus.
 10. **Journalisation** : valeur mesurée **et** seuil appliqué pour chaque filtre, à chaque évaluation. Sans le seuil appliqué, la révision post-500-sessions est impossible.
 11. **Tests** : **T-P1 un spread de -0,01 est refusé (R21)** ; T-P2 un saut d'ask au p95 mesuré passe, un saut au-delà est refusé (R17) ; T-P3 la chute d'ask utilise le q95 en ligne, pas 0,03 (R18) ; **T-P4 profondeur absente $\Rightarrow$ refus, toujours (R03)** ; T-P5 la garde de vitesse est symétrique : une pente de +0,50 \$/s et de -0,50 \$/s sont **toutes deux** refusées (R23) ; T-P6 un krach d'ask de 0,279 s est détecté à 4 Hz et manqué à 1 Hz (R31).
 12. **Critères de validation** : taux de refus par test entre 5 % et 30 % ; aucun à 0 % ni à 100 %.
-

9. `Regime`

1. **Nom** : `Regime` (garde d'universalité de la volatilité)
 2. **Objectif** : empêcher l'application d'un calibrage de régime calme à un régime agité.
 3. **Responsabilités** : médiane glissante de `$\sigma_{30}$` sur les 20 sessions précédentes ; ratio de régime ; volatilité annualisée implicite ; verrouillage de session hors bornes.
 4. **Données reçues** : `$\sigma_{30}$` de la session en cours et historique des 20 précédentes.
 5. **Données produites** : ratio, verdict, volatilité annualisée implicite.
 6. **Fonctions publiques** : `regime_ratio(sigma_session)` , `regime_ok(ratio)` , `implied_annual_vol(sigma_30)` , `record_session(sigma_30)` .
 7. **Dépendances** : `Oracle` , `Chronicle` .
 8. **Règles métier** : ratio `$\in [0,4 ; 2,5]$` , évalué à `$\tau = 30$  s` puis à chaque tick ; hors bornes $\Rightarrow$ **aucune cotation sur la session**, alerte, et **révision obligatoire de** `$\delta_{guard}$` **avant reprise** (R35) ; historique insuffisant (< 20 sessions) $\Rightarrow$ fail-closed.
 9. **Cas d'erreur** : historique insuffisant ; `$\sigma_{30}$` de référence indisponible.
 10. **Journalisation** : ratio et volatilité annualisée implicite **par session**, obligatoire.
 11. **Tests** : **T-RG1 le corpus à 6,9 % de volatilité annualisée est marqué non représentatif face à une référence à 45 % (R35)** ; T-RG2 historique < 20 sessions $\Rightarrow$ refus ; T-RG3 le ratio est insensible au signe (test miroir).
 12. **Critères de validation** : volatilité annualisée implicite publiée pour chaque session sur 500 sessions.
-

10. `Warden`

1. **Nom** : `Warden` (blocs de validité)
2. **Objectif** : la pile de vetos fail-closed, avec un code atomique unique par test.
3. **Responsabilités** : vérifications de session, de régime, de fraîcheur, d'intégrité, de qualité de carnet, d'état interne ; ordre d'évaluation déterministe ; arrêt au premier faux.
4. **Données reçues** : tout ce que produisent `Anchor` , `Oracle` , `Garnet` , `Prism` , `Regime` , plus un `PositionSnapshot` en lecture seule.
5. **Données produites** : `GateResult` = {ok, premier code faux, valeurs mesurées de tous les tests évalués}.
6. **Fonctions publiques** : `integrity_ok(ctx)` , `freshness_ok(ctx)` , `session_ok(ctx)` , `state_ok(ctx)` , `evaluate_all(ctx) -> GateResult` .
7. **Dépendances** : `Anchor` , `Oracle` , `Garnet` , `Prism` , `Regime` , `Positions` (lecture seule).
8. **Règles métier** : **fail-closed partout, jamais fail-open** ; invariant de complémentarité ≤ 1 tick (R32) ; basis dans la bande à 3σ (R22) ; `HALT` absent $\Rightarrow$ refus (R05) ; `D_max` en **coupe-circuit** (R19) et sa position dans la conjonction est contestée par R38 ; test d'espérance nette à la place de `P_max` (R20) ; T10 et T11 journalisés comme **une seule dimension** (R50).
9. **Cas d'erreur** : contexte incomplet $\Rightarrow$ refus ; deux codes vrais simultanément $\Rightarrow$ assertion, car le code doit être atomique.
10. **Journalisation** : le premier code faux, plus **toutes** les valeurs mesurées des tests déjà évalués. La distribution des non-entrées est la donnée qui permettra la révision.
11. **Tests** : T-W1 sur le corpus, la conjonction complète ne donne pas 0 entrée (régression contre R16) et aucun test ne refuse 100 % ; T-W2 le basis unilatéral n'existe plus, seule la bande à 3σ subsiste (R22) ; T-W3 les trois seuils de fraîcheur sont 2,5 / 8 / 3 s, pas 3 / 7 s (R33, R44) ; T-W4 **INVARIANT_ROMPU déclenche à 1,01 tick et pas à 1,00 tick (R32)** ; T-W5 `D_max` n'apparaît pas comme condition d'entrée (R19), et le comportement en phase [240;270[est explicitement testé (R38) ; T-W6 `P_max` et ses trois constantes sont absentes de la base de code (R20) ; T-W7 `$\Phi_{30}$` est absent (R11).
12. **Critères de validation** : zéro cotation sur données incohérentes ; taux de refus par code cohérent avec le corpus (invariant $\approx 0,2$ %, spread ≈ 14 %) ; PnL 0,00 \$ dans chaque sous-groupe en mode veto.

11. `Signal`

1. **Nom** : `Signal` (moteur d'entrée)
2. **Objectif** : produire une **intention** d'entrée, jamais un ordre.
3. **Responsabilités** : évaluer la conjonction d'entrée ; calculer le prix limite et la taille ; déterminer le côté par miroir ; fixer la validité à 2 s.
4. **Données reçues** : `GateResult` , `MeasureSet` , `ObservedTerms` , `SizeAdvice` de `Risk` .
5. **Données produites** : `OrderIntent` = {side, limit_price, size, ttl, reason, δ appliquée, post_only} ou `NoEntry(code)` .

6. **Fonctions publiques** : `evaluate(ctx) -> Option<OrderIntent>`, `limit_price(anchored, delta, book, mode)`, `eligible(ctx)`.
7. **Dépendances** : `Warden`, `Oracle`, `Garnet`, `Risk`, `Chronos`.
8. **Règles métier** : fenêtre `[180;270]` ; $z \geq 1,0$; $X_{60} = 0$; $P \geq 5$ s ; $e > \delta_{\min}$ depuis ≥ 2 s ; **règle de prix : deux modes derrière un drapeau, `PASSIVE_ONLY` (option A) et `TAKER_ASSUMED` (option B), et `TAKER_ASSUMED` est refusé au démarrage tant que $\Phi(z)$ n'est pas calibré par sous-groupe. Voir R36.** ; taille `min(floor(N), profondeur)` ; jamais de consommation de `p`, `D` signé, `lag`, pente signée.
9. **Cas d'erreur** : $\delta_{\min} > \delta_{\max}$; prix limite hors `[0;1]` ; taille `< PARTS_MIN` ; mode `TAKER_ASSUMED` non autorisé.
10. **Journalisation** : `OrderIntent` complet ou `NoEntry` avec code atomique ; δ appliquée et le palier dont elle vient.
11. **Tests** : T-S1 le nombre d'événements éligibles par session est publié et $\geq 0,5$ en moyenne (R16) ; T-S2 aucune entrée hors `[180;270]` (R26) ; T-S3 symétrie miroir exacte : sous jeu de données miroir, `|m*`, `z`, `$\sigma_T$` , `$\delta$` , `e`, `N` identiques et prix complémentaire ; **T-S4 en mode `PASSIVE_ONLY`, le prix limite ne dépasse jamais le bid du côté (R36)** ; T-S5 δ appliquée est toujours $\geq \delta_{\min}$ et multiple du tick (R37) ; T-S6 `TAKER_ASSUMED` refuse de démarrer sans calibration validée.
12. **Critères de validation** : répartition des sessions par couleur publiée ($X_{60} = 0$ contre `> 0`) ; $\geq 0,5$ événement éligible par session, sinon le mécanisme ne passe pas à l'échelle.

12. `Exit`

1. **Nom** : `Exit` (moteur de sortie)
2. **Objectif** : produire une **intention** de sortie ou un état de suspension, jamais un gain.
3. **Responsabilités** : V0 à V8 selon l'ordre d'évaluation imposé ; verrou terminal ; états de suspension ; **jamais `HOLD` silencieux quand la sortie était impossible.**
4. **Données reçues** : `PositionSnapshot`, `MeasureSet`, `ObservedTerms`, frais de sortie, `$\tau$` .
5. **Données produites** : `ExitDecision` $\in \{ \text{SELL_VALUE}, \text{SELL_INVALIDATION}, \text{SELL_PROFIT}, \text{HOLD}, \text{HOLD_TO_SETTLEMENT}, \text{SUSPEND_DATA}, \text{SUSPEND_LIQUIDITY}, \text{SETTLE}, \text{NO_POSITION} \}$.
6. **Fonctions publiques** : `decide(ctx) -> ExitDecision` ; `v0_state_valid`, `v1_value`, `v2_persistent_invalidation`, `v3_margin_erosion`, `v4_vol_explosion`, `v5_profit_take`, `v6_terminal_lock`, `v7_liquidity`, `v8_vanishing_bid`.
7. **Dépendances** : `Warden`, `Oracle`, `Garnet`, `Fees`, `Positions`, `Chronos`.
8. **Règles métier** : **politique de sortie derrière un drapeau unique, `SETTLE_ONLY` (étape 4) ou `EARLY_EXIT` (étape 5). Voir R42.** ; ordre d'évaluation strict et non réorganisable ; seuils de fraîcheur de l'étape 4 (R44) ; borne basse de spread obligatoire (R45) ; `$\tau_{\text{lock}} = 262$  s` et non 240 s (R43) ; **V1 et V5 désactivées** tant que `q` n'est pas calibré (R46) ; **0,92 interdit en dur** (R47) ; V3 diagnostic seulement ; V4 jamais déclencheur autonome ; `q = 1 - p` obligatoire pour une position NO.
9. **Cas d'erreur** : position inexistante ; bid absent ; remplissage non confirmable ; `q` non disponible alors qu'une règle l'exige ($\Rightarrow$ refus, jamais valeur par défaut).
10. **Journalisation** : à chaque décision, `$\tau$` , side, `K`, `p`, `q`, bid, ask, spread, frais estimés, `V_hold`, `D_sell`,

signal actif, raison principale, résultat du remplissage, **et résultat contrefactuel à l'échéance**.

11. **Tests** : T-X1 les 7 contradictions interdites lèvent une assertion ; T-X2 `SELL_FILLED` **n'est jamais produit sans confirmation de fill (étape 5 §4.3)** ; T-X3 une position NO utilise $q = 1 - p$ (R46) ; T-X4 `$\tau\_lock$` est 262 s (R43) ; T-X5 V1 et V5 sont inertes tant que la calibration manque (R46) ; T-X6 la constante 0,92 est absente du code (R47) ; T-X7 V4 seul ne produit jamais de vente ; T-X8 un bid absent produit `SUSPEND_LIQUIDITY` et jamais `HOLD` .
12. **Critères de validation** : sous `SETTLE_ONLY` , zéro sortie anticipée hors sécurité. Sous `EARLY_EXIT` , comparaison obligatoire contre `HOLD_TO_SETTLEMENT` sur ≥ 29 sessions indépendantes réparties UP et DOWN.

13. `Risk`

1. **Nom** : `Risk`
2. **Objectif** : dimensionner, borner la capacité et couper.
3. **Responsabilités** : `N` ; `PARTS_MIN` ; plafond de profondeur ; budget de mesure de M3 ; kill-switch ; détection d'adaptation de la contrepartie.
4. **Données reçues** : `ask` , frais, profondeur, `PositionSnapshot` , historique de taux de fill.
5. **Données produites** : `SizeAdvice` = {n, plafonnée_par, refusée_pourquoi}, état du kill-switch.
6. **Fonctions publiques** : `size(ask, depth)` , `capacity_ok(size)` , `measurement_budget_left()` , `kill_switch_state()` , `detect_adaptation(fill_rate_window)` .
7. **Dépendances** : `Garnet` , `Fees` , `Positions` , `Kernel` .
8. **Règles métier** : `C = 5,00 $` , `PARTS_MIN = 5` , **une seule formule de sizing** (R40) ; `CAPITAL_DEMI` et `RATIO_PLEIN` **absents** (R30) ; contrainte induite `ask  $\leq$  0,98926` environ ; capacité : 5,00 \$ = 8,7 % d'un gros ordre médian (57,64 \$) et 0,023 % du notionnel de session (21 590 \$), donc impact négligeable **par construction** ; budget de mesure M3 : ≤ 400 \$ sur 100 sessions ; **signe d'adaptation** = baisse du taux de fill à δ constant sur 20 sessions glissantes, ou quote placée systématiquement 1 tick devant celle du bot $\Rightarrow$ augmenter δ d'un tick, et si le fill ne remonte pas sur 20 sessions, **retrait du mécanisme**.
9. **Cas d'erreur** : profondeur inconnue $\Rightarrow$ taille 0 ; `N < 5` $\Rightarrow$ refus ; budget épuisé $\Rightarrow$ refus ; kill-switch armé $\Rightarrow$ refus.
10. **Journalisation** : `N` , ce qui l'a plafonné, budget restant, état du kill-switch à chaque décision.
11. **Tests** : T-R1 **une seule formule de sizing existe** (R40) ; T-R2 `CAPITAL_DEMI` et `RATIO_PLEIN` absents du code (R30) ; T-R3 `ask = 0,99` $\Rightarrow$ `N < 5` $\Rightarrow$ refus ; T-R4 profondeur inconnue $\Rightarrow$ taille 0 ; T-R5 le budget de mesure borne l'exposition à 400 \$; T-R6 la détection d'adaptation déclenche sur une baisse simulée du taux de fill.
12. **Critères de validation** : ratio PnL / capital engagé $> +1,0$ % par événement rempli, **dans chaque sous-groupe**.

14. `Arbiter`

1. **Nom** : `Arbiter` (module de décision)

2. **Objectif** : produire **exactement une** décision par événement, et traiter entrée et sortie comme un **problème de double arrêt**, pas comme deux filtres empilés.
3. **Responsabilités** : priorités ; idempotence ; validation finale avant envoi ; assertions d'incohérence ; déduplication des intentions.
4. **Données reçues** : `OrderIntent` de `Signal`, `ExitDecision` de `Exit`, `SizeAdvice`, `PositionSnapshot`, `GateResult`.
5. **Données produites** : `Action` $\in \{ \text{CANCEL_ALL}, \text{CANCEL_ONE}, \text{SUBMIT_ENTRY}, \text{SUBMIT_EXIT}, \text{HOLD}, \text{SUSPEND}, \text{REFUSE} \}$ avec sa raison unique.
6. **Fonctions publiques** : `decide(ctx) -> Action`, `assert_consistency(action, ctx)`, `is_duplicate(action)`.
7. **Dépendances** : `Signal`, `Exit`, `Risk`, `Positions`, `Warden`, `Chronicle`.
8. **Règles métier : ordre de priorité absolu** : (1) `TRAVERSEE_SEUIL` $\Rightarrow$ `CANCEL_ALL` avant tout autre traitement, (2) rupture d'intégrité $\Rightarrow$ `CANCEL_ALL` + `SUSPEND`, (3) sortie de sécurité, (4) sortie discrétionnaire, (5) entrée ; **une seule action par événement ; jamais d'entrée s'il existe une position ou un ordre en vol** ; les 7 contradictions de l'étape 5 sont des **assertions d'exécution**.
9. **Cas d'erreur** : deux actions candidates de même priorité $\Rightarrow$ assertion ; action incohérente avec l'état $\Rightarrow$ refus et alerte.
10. **Journalisation** : l'action retenue, sa raison, **et les actions candidates écartées avec leur raison**.
11. **Tests** : T-AR1 `TRAVERSEE_SEUIL` **a la priorité sur toute autre action (R32)** ; T-AR2 zéro double ordre sur 10 000 événements rejoués, y compris sous événements dupliqués et désordonnés ; T-AR3 idempotence : deux événements identiques produisent une seule action ; T-AR4 chacune des 7 contradictions interdites lève ; T-AR5 aucune entrée avec position ouverte.
12. **Critères de validation** : zéro état incohérent sur un rejeu de 500 sessions ; 100 % des actions traçables à une raison unique.

15. `Executor`

1. **Nom** : `Executor`
2. **Objectif** : être le **seul** émetteur d'ordres, et rendre le double ordre structurellement impossible.
3. **Responsabilités** : cycle de vie complet (envoi, ack, fill, partiel, annulation, rejet) ; `post_only` ; verrou d'unicité ; validité 2 s ; annulation ≤ 500 ms ; **un seul réessai** puis suspension 10 s ; capture des 8 horodatages.
4. **Données reçues** : `Action` validée.
5. **Données produites** : `OrderLifecycle` = {order_id, état, prix et qté exécutés, maker/taker, code de rejet, 8 horodatages}.
6. **Fonctions publiques** : `submit(intent)`, `cancel(order_id)`, `cancel_all()`, `on_ack`, `on_fill`, `on_partial`, `on_reject`, `in_flight_count()`.
7. **Dépendances** : `Arbiter`, `Positions`, `Chronicle`, `Kernel`.
8. **Règles métier : verrou d'unicité** : `in_flight_count() > 0` **interdit tout nouvel envoi** ; en mode `PASSIVE_ONLY`, le drapeau `post_only` est **obligatoire** et un rejet pour croisement est un **succès attendu**, pas

une erreur (R36) ; annulation puis recotation, **jamais de modification en place** ; le gain n'est compté **qu'après confirmation du fill** ; un rejet compte l'événement comme **non atteignable** ; une exécution partielle **mesure la profondeur au meilleur niveau**, donc elle est journalisée comme une mesure et non comme un incident.

9. **Cas d'erreur** : ack manquant après N ms ; fill sans ordre connu ($\Rightarrow$ alerte critique) ; annulation non acquittée ; rejet répété ; déconnexion ($\Rightarrow$ `cancel_all` puis suspension).
 10. **Journalisation** : les 8 horodatages sur **chaque** ordre, plus `order_id`, `maker/taker`, code de rejet.
 11. **Tests** : T-E1 les 8 horodatages sont présents et croissants sur chaque ordre (R07, R08) ; T-E2 en `PASSIVE_ONLY`, aucun ordre ne traverse le spread (R36) ; T-E3 zéro double ordre sous rafale de 1 000 événements en 1 s ; T-E4 délai d'annulation ≤ 500 ms au P90 et zéro fill sur un ordre dont le côté ancré a changé (R32, garde binaire) ; T-E5 le budget de latence est calculable depuis le journal ; T-E6 un rejet donne un seul réessai puis 10 s de suspension.
 12. **Critères de validation** : zéro fill sur ordre périmé. Un seul suffit à invalider la garde.
-

16. `Positions`

1. **Nom** : `Positions`
 2. **Objectif** : détenir l'état interne vérité et le réconcilier avec l'API.
 3. **Responsabilités** : position ouverte ; nombre d'ordres en vol ; fills partiels agrégés ; réconciliation ; issue et PnL contrefactuel ; affectation du sous-groupe.
 4. **Données reçues** : `OrderLifecycle`, état API, `issue_officielle`.
 5. **Données produites** : `PositionSnapshot` (lecture seule), PnL réalisé, PnL contrefactuel, sous-groupe.
 6. **Fonctions publiques** : `snapshot()`, `apply(lifecycle_event)`, `reconcile(api_state)`, `settle(outcome)`, `subgroup()`.
 7. **Dépendances** : `Kernel`, `Chronicle`.
 8. **Règles métier** : une position fantôme non réconciliée vaut refus ; réconciliation avant chaque décision ; PnL calculé avec `fee_settlement`, jamais `fee_sizing` (R41) ; sous-groupe = (direction de l'oracle) $\times$ (côté gagnant), quatre valeurs, obligatoire sur chaque événement.
 9. **Cas d'erreur** : divergence état local et API ; issue absente ; fill non attribuable.
 10. **Journalisation** : chaque transition d'état ; chaque divergence de réconciliation ; issue et sous-groupe à la clôture.
 11. **Tests** : T-PO1 une divergence de réconciliation bloque toute décision (R04) ; T-PO2 les fills partiels s'agrègent correctement ; T-PO3 le sous-groupe est renseigné sur 100 % des événements (R09) ; T-PO4 le PnL utilise `fee_settlement` (R41).
 12. **Critères de validation** : réconciliation à 100 %, zéro position fantôme sur 500 sessions.
-

17. `Intake`

1. **Nom** : `Intake`

2. **Objectif** : être la seule frontière avec le monde extérieur, et livrer des snapshots propres, ordonnés et horodatés.
3. **Responsabilités** : abonnements aux quatre flux ; déduplication ; détection de désordre ; booléens de disponibilité ; mesure d'âge et de trou ; **déclenchement par l'union des événements** ; horloges séparées.
4. **Données reçues** : flux oracle, carnet, spot, trades ; payload officiel.
5. **Données produites** : `ValidatedEvent` typé par flux, avec disponibilité, âge, trou et compteur d'anomalies.
6. **Fonctions publiques** : `subscribe_all()` , `next_event()` , `age(stream)` , `gap(stream)` , `anomaly_counters()` .
7. **Dépendances** : `Kernel` , `Chronos` , `Chronicle` .
8. **Règles métier** : **4 Hz minimum sur le carnet, union des événements oracle et carnet (R31)** ; doublon (même `t_ms` et même charge) ignoré, compté, journalisé, et **ne fait avancer aucun compteur de persistance** ; `t_ms` régressif rejeté sur les flux monotones, snapshot ignoré sur le carnet, **aucun recalcul rétroactif d'une décision déjà prise** ; `ts_src` non numérique $\Rightarrow$ champ marqué inutilisable, jamais deviné (R10) ; `trades.direction` livré brut, **consommation interdite (R11)**.
9. **Cas d'erreur** : déconnexion ($\Rightarrow$ `cancel_all` + suspension) ; charge malformée ; horodatage absent ; dérive d'horloge détectée par le basis.
10. **Journalisation** : compteurs de doublons, de désordres et de malformés par session ; cadence effective par flux.
11. **Tests** : T-I1 un doublon n'avance aucun compteur ; T-I2 un `t_ms` régressif est rejeté et ne recalcule rien ; **T-I3 la cadence effective du carnet est ≥ 4 Hz et le krach de 0,279 s est vu (R31)** ; T-I4 aucune sentinelle numérique ne sort d' `Intake` (R21) ; **T-I5 `ts_src` non numérique est marqué inutilisable et jamais employé comme heure (R10)** ; T-I6 une déconnexion déclenche `cancel_all` .
12. **Critères de validation** : cadence effective mesurée et publiée par flux ; zéro décision prise sur un événement désordonné.

18. `Harness`

1. **Nom** : `Harness` (rejeu et dry run)
2. **Objectif** : rejouer des sessions enregistrées de façon **bit-à-bit déterministe**, et faire tourner M3.
3. **Responsabilités** : lecture de CSV et de journaux ; horloge simulée ; simulateur de moteur d'appariement paramétrable ; génération de **jeux de données miroir** pour les tests de non-directionnalité ; balayage de δ pour M3 ; agrégation du PnL **par sous-groupe** ; balayage de configurations.
4. **Données reçues** : sessions enregistrées, configurations à balayer.
5. **Données produites** : rapport **par session** (pas seulement par seconde) ; PnL par sous-groupe ; taux de fill par phase et par δ ; table de décote reconstruite ; IC de Clopper-Pearson.
6. **Fonctions publiques** : `replay(session, config)` , `mirror(session)` , `sweep(configs)` , `subgroup_pnl(results)` , `fill_rate_table(results)` , `clopper_pearson(k, n)` .
7. **Dépendances** : tous les modules.

8. **Règles métier** : le simulateur de fill **ne doit jamais supposer une profondeur inconnue** : sans L2, il ne peut produire qu'un fill au ask ou un contact BBO, et **les deux doivent être étiquetés différemment** (c'est le cœur de R36 et R08) ; un contact BBO n'est **jamais** compté comme un fill ; le rejeu est déterministe ; **jamais de résultat agrégé sans la ventilation par sous-groupe**.
9. **Cas d'erreur** : session incomplète ; issue manquante ; non-déterminisme détecté entre deux rejeux.
10. **Journalisation** : configuration, graine, version des paramètres, hash du jeu de données.
11. **Tests** : T-H1 deux rejeux identiques donnent un résultat identique bit-à-bit ; **T-H2 sur un jeu de données miroir, le PnL est le miroir exact (test de non-directionnalité opérationnel)** ; T-H3 un contact BBO n'est jamais compté comme un fill (R08) ; T-H4 le balayage de δ produit la table par phase (R08) ; T-H5 aucun agrégat n'est publiable sans ses 4 sous-groupes (R09) ; T-H6 l'IC de Clopper-Pearson d'un 3/3 rend bien [29,2 % ; 100 %].
12. **Critères de validation** : rejeu du corpus reproduisant les chiffres clés de l'étape 4 (0 entrée sous la configuration de l'étape 3 ; lag médian ≤ 0 ; σ_{30} médian $\approx 7,20$ \$). Si le rejeu ne reproduit pas ces trois-là, **le harnais est faux avant la stratégie**.

19. `Gate`

1. **Nom** : `Gate` (contrôles de passage en live)
2. **Objectif** : refuser le démarrage en live tant que les onze critères de 2.11 ne sont pas verts, et **retirer** un mécanisme dès qu'un critère de retrait se déclenche.
3. **Responsabilités** : évaluation des 11 critères ; verrouillage des drapeaux de mode ; versionnage des paramètres et des coefficients de Platt ; déclenchement du retrait.
4. **Données reçues** : rapports du `Harness`, agrégats du `Chronicle`.
5. **Données produites** : `LiveReadiness` = {critère, statut, valeur mesurée, seuil}, et un verdict binaire.
6. **Fonctions publiques** : `evaluate_readiness()`, `may_go_live()`, `may_enable(mode)`, `withdrawal_triggered()`.
7. **Dépendances** : `Harness`, `Chronicle`.
8. **Règles métier** : **un seul sous-groupe négatif $\Rightarrow$ retrait immédiat** ; si aucune valeur de `z_min`, `$\delta_{guard}$` ou `D_max` ne garde les quatre sous-groupes positifs, le mécanisme est **retiré, pas réajusté** ; la révision retient la valeur qui garde le PnL positif **dans chaque** sous-groupe, jamais celle qui maximise l'agrégat ; `p` et `D` ne rentrent que si `Brier(p)  $\leq$  Brier(mid)` dans **chacun** des quatre sous-groupes sur ≥ 50 sessions étiquetées, coefficients versionnés ; un seul sous-groupe en échec $\Rightarrow$ `p` reste un score **définitivement**.
9. **Cas d'erreur** : données insuffisantes pour évaluer un critère $\Rightarrow$ critère **rouge**, jamais neutre.
10. **Journalisation** : le tableau des 11 critères à chaque évaluation, avec valeur et seuil, et l'historique des verdicts.
11. **Tests** : T-G1 un critère non évaluable est rouge ; T-G2 un sous-groupe négatif déclenche le retrait ; T-G3 `may_go_live()` est faux si un seul champ officiel manque ; **T-G4 `p` ne peut pas être activé avec un Brier défavorable dans un seul sous-groupe (R24)** ; T-G5 la révision de paramètres refuse une valeur qui maximise l'agrégat au détriment d'un sous-groupe.

3.3 Ce que l'architecture rend structurellement impossible

Erreur	Mécanisme de prévention
Double ordre	Verrou d'unicité dans <code>Executor</code> + <code>Arbiter</code> qui ne produit qu'une action + T22 dans <code>Warden</code>
Fail-open sur champ manquant	Booléens de disponibilité dans <code>Kernel</code> , aucune sentinelle numérique, test statique T-K3
Mélange des deux horloges	Types incompatibles dans <code>Kernel</code> , échec à la compilation
Réutilisation croisée des modèles de frais	Trois fonctions nommées dans <code>Fees</code> , test statique T-F1
Deux définitions de σ_{30}	Une seule implémentation dans <code>Oracle</code> , test statique T-O3
Constante gelée là où une mesure est requise	Tests statiques T-O1, T-GA4
Consommation d'une grandeur directionnelle	Tests statiques T-A3, T-CH5, T-GA6, T-W7
Décision non traçable	<code>Chronicle</code> en rang 1, un échec d'écriture arrête le bot
Ordre agressif en mode passif	<code>post_only</code> obligatoire, T-S4 et T-E2
Contact BBO compté comme un fill	<code>Harness</code> étiquette les deux différemment, T-H3
Agrégat publié sans ventilation par sous-groupe	T-H5 et <code>Gate</code>

4. Séquence d'exécution à chaque tick

Déclencheur : l'union des événements oracle et carnet (R31). Jamais une horloge fixe. La cadence effective est donc événementielle, avec un plancher de 4 Hz sur le carnet, sachant que la cadence médiane mesurée du carnet est de **0,013 s** et celle de l'oracle de **1 s** : le carnet peut bouger environ **77 fois** entre deux événements oracle.

Deux boucles imbriquées, avec des grandeurs distinctes :

- **Boucle oracle (1 Hz)** : `$\sigma_{30}$` , `$\sigma_T$` , `$\lambda$` , `$m^*$` , `$z$` , `côté`, `p_ancré`, `$X_{60}$` , `P`, `v`, `TWAP30`, `$w(\tau)$` .
- **Boucle carnet (4 Hz minimum)** : `ask`, `bid`, `spread`, `e`, `méd3s`, `q95_saut`, `q95_chute`, `invariants`, `N`, `$\delta$` .

4.1 Les 17 étapes

1. Réception de la donnée (`Intake`)

Un événement arrive sur l'un des quatre flux. Il est typé, horodaté sur son horloge de capture propre, et accompagné de ses booléens de disponibilité. `t_decision` **n'est pas encore posé**.

2. Vérification de fraîcheur et de validité (`Intake`, `Warden`)

Contrôle	Seuil	Code
Âge du dernier tick oracle	$\leq 2,5\text{ s}$	<code>TICK_PERIME</code>
Trou du flux oracle	$\leq 8\text{ s}$	<code>TICK_TROU</code>
Trou du flux carnet	$\leq 3\text{ s}$	<code>TICK_TROU</code>
Quatre champs BBO présents	Booléen, jamais sentinelle	<code>SPREAD_INDISPO</code>
<code>HALT</code> présent et faux	—	<code>HALT</code>

Un trou d'oracle **gèle le compteur de persistance**, il ne le remet pas à zéro : la distinction évite qu'une coupure de flux soit lue comme un changement de régime.

3. Normalisation et nettoyage (`Intake`)

Déduplication (même `t_ms` et même charge) ; détection de `t_ms` régressif ; conversion vers les types du `Kernel` ; **séparation stricte des deux horloges**. `ts_src` non numérique est marqué inutilisable et **jamais** employé comme heure (R10). `trades.direction` est transmis brut, avec interdiction de consommation (R11).

4. Calcul des mesures brutes (`Oracle`, `Garnet`)

Sur événement oracle : `$\sigma_{30}$` par l'estimateur A sur la fenêtre `$]\tau-30 ; \tau]$` , puis `$\sigma_T$` , `$\lambda$` , `$m^*$` , `TWAP30`. Refus si la fenêtre de 30 s n'est pas pleine ou si `$\sigma_{30} \notin [0,5 ; 40] \$$` → `SIGMA_INDISPO`.

Sur événement carnet : `spread`, `mid`, `méd3s(ask)`, `q95(|Δask|_3s)`, `q95(chute_3s)`, les deux invariants, `basis` apparié `as-of` et sa bande glissante.

5. Calcul des termes observés (`Oracle`, `Garnet`)

`z = |m*|/σT` ; `côté = miroir(signe(m*))` ; `p_ancré = Φ(z)` ; `e = |p_ancré - ask_côté|` ; `X60` ; `P` ; `v` ; `δmin`, `δguard`, `δ` ; `N`. Le prix du côté NON est **dérivé** du côté OUI par l'invariant, pas lu séparément.

6. Filtres de microstructure (`Prism`)

`spread ∈ [0 ; 0,04]` bornes strictes des deux côtés ; anti-couteau `ask ≥ méd3s - q95(chute)` ; stabilité `max(|Δask|_3s) ≤ q95(saut)` ; vitesse symétrique `v ≤ 0,45 $/s` ; profondeur `≥ floor(N)` (**fail-closed permanent**).

7. Évaluation des blocs de validité (`Warden`)

Ordre déterministe, **arrêt au premier faux**, code atomique unique : session, régime, fraîcheur, intégrité, qualité de carnet, état interne. Les valeurs mesurées de tous les tests déjà évalués sont conservées pour la journalisation, même en cas de refus précoce.

8. Recherche d'un signal (`Signal`)

Seulement si l'étape 7 est verte. Conjonction : `180 ≤ τ ≤ 270` ; `z ≥ 1,0` ; `X60 = 0` ; `P ≥ 5 s` ; `e > δmin` **continu depuis ≥ 2 s** ; test d'espérance nette ; coupe-circuit `Dmax` (**position contestée**, R38). Produit une **intention**, pas un ordre.

9. Vérification des positions existantes (`Positions`, `Exit`)

Réconciliation avec l'API **avant** toute décision. S'il existe une position : le moteur de sortie s'exécute et l'entrée est structurellement interdite (T22). S'il existe un ordre en vol : aucun nouvel envoi (verrou d'unicité). Divergence ⇒ refus et alerte.

10. Vérification du risque (`Risk`, `Fees`)

`N = floor(C/(ask + fee_sizing(ask) + 0,01))`, plafonné par la profondeur, plancher `PARTS_MIN = 5`. Garde de pré-vol sur `fee_bps`. Budget de mesure de M3. État du kill-switch.

11. Génération d'une décision (`Arbiter`)

Une seule action, selon la priorité absolue :

1. `TRAVERSEE_SEUIL` ⇒ `CANCEL_ALL`, **avant tout autre traitement**
2. Rupture d'intégrité ⇒ `CANCEL_ALL` + `SUSPEND`
3. Sortie de sécurité (`SUSPEND_DATA`, `SUSPEND_LIQUIDITY`)
4. Sortie discrétionnaire (seulement si le mode `EARLY_EXIT` est actif)
5. Entrée
6. `HOLD`

12. Validation finale avant envoi (`Arbiter`)

Les 7 assertions d'incohérence de l'étape 5 ; cohérence action / état ; prix dans `[0;1]` et multiple du tick ; taille `≥ PARTS_MIN` ; en mode `PASSIVE_ONLY`, **vérification que le prix limite ne dépasse pas le bid du côté** (R36) ; déduplication d'intention. `t_decision` est posé ici.

13. Création, modification ou annulation (`Executor`)

Envoi avec `post_only` obligatoire en mode passif, validité 2 s. **Jamais de modification en place : annulation puis recotation.** Recotation déclenchée par $|z(\tau) - z(\tau_{\text{envoi}})| \cdot d\Phi/dz > \delta$, par un saut d'ask $> q95_{\text{saut}}$, ou **systématiquement au franchissement du palier $\tau = 240$ s** même si `z` n'a pas bougé, parce que `delta` change. `t_envoi` est posé.

14. Suivi de l'exécution (`Executor`)

`t_ack`, `t_fill`, prix et quantité exécutés, `maker/taker`, codes de rejet. Une exécution partielle est journalisée comme une **mesure de profondeur au meilleur niveau**. Un rejet compte l'événement comme **non atteignable**, avec un seul réessai puis 10 s de suspension. Un rejet `post_only` pour croisement est un **succès attendu**, pas une erreur.

15. Mise à jour de l'état interne (`Positions`)

Agrégation des fills, mise à jour de `position` et `n_ordres_en_vol`, réconciliation, affectation du sous-groupe. À la clôture : issue, PnL réalisé avec `fee_settlement`, et **PnL contrefactuel** de la branche non prise.

16. Journalisation complète (`Chronicle`)

24 champs + 8 horodatages + sous-groupe + code atomique + **valeur mesurée et seuil appliqué pour chaque test évalué** + actions candidates écartées. Écriture append-only. **Un échec d'écriture arrête le bot.** Un refus s'écrit ; une attente ne s'écrit pas, parce qu'elle n'existe pas.

17. Gestion des erreurs et des situations dégradées

Voir 4.2. Principe : **toute dégradation est fail-closed, journalisée, et annule les ordres en vol.** Aucune dégradation silencieuse.

4.2 Situations dégradées, ligne par ligne

Situation	Détection	Action immédiate	Reprise
Donnée absente : champ BBO manquant	Booléen de disponibilité faux	<code>SPREAD_INDISPO</code> , annulation, aucune cotation	Dès snapshot complet et cohérent
Donnée absente : <code>K</code> ou <code>endDate</code>	À l'ouverture	Session entière refusée, une seule ligne de refus écrite	Session suivante
Donnée absente : profondeur L2 ou réconciliation	À chaque décision	Fail-closed permanent : le bot ne cote jamais	Quand le champ existe
Donnée incohérente : invariant de complémentarité > 1 tick	Chaque snapshot carnet	<code>INVARIANT_ROMPU</code> , annulation	Retour dans le tick
Donnée incohérente : spread négatif (carnet croisé)	Chaque snapshot	<code>SPREAD</code> , refus. Jamais interprété comme une opportunité	$s \in [0 ; 0,04]$

Donnée incohérente : basis hors bande à 3σ	Chaque ligne spot	<code>BASIS_CORROMPU</code> , suspension de la session. Signature probable d'une désynchronisation d'horloge	Retour dans la bande
Donnée retardée : âge oracle > 2,5 s	Chaque évaluation	<code>TICK_PERIME</code> , annulation. L'ancrage est la seule référence de prix : sans oracle frais, le bot n'a pas de prix	Âge $\leq 2,5$ s
Donnée retardée : trou oracle > 8 s ou carnet > 3 s	Continu	<code>TICK_TROU</code> , annulation, gel du compteur de persistance	Reprise des deux flux
Donnée dupliquée	Même <code>t_ms</code> et même charge	Ignorée, comptée, journalisée. N'avance aucun compteur de persistance	Immédiate
Donnée désordonnée : <code>t_ms</code> régressif	Comparaison au dernier <code>t_ms</code>	Flux monotone (oracle) : rejet. Carnet : snapshot ignoré, compteur d'anomalies. Aucun recalcul rétroactif d'une décision déjà prise	Prochain événement croissant
Données contradictoires : oracle et spot divergent	Bande de basis	Le basis arbitre. L'oracle n'est jamais corrigé par le spot : il est la source unique de résolution	Retour dans la bande
Traversée du strike avec ordre en vol	<code>signe(m*(τ)) $\neq$ signe(m*(τ_{envoi}))</code>	Annulation immédiate et inconditionnelle, avant tout autre traitement. Budget cible < 300 ms. Puis <code>X₆₀ := 1</code>	60 s minimum, soit en pratique la fin de session. C'est voulu
Rejet d'ordre	Réponse du moteur	Journaliser le code, un seul réessai, puis suspension 10 s. Événement compté non atteignable	Après 10 s
Exécution partielle	<code>on_partial</code>	Journaliser la quantité. C'est une mesure de la	Continu

		profondeur , le champ manquant n°1	
Ack manquant	Délai dépassé	<code>cancel</code> puis réconciliation forcée	Après réconciliation
Fill sans ordre connu	Réconciliation	Alerte critique, kill-switch, aucune nouvelle décision	Intervention manuelle
Déconnexion	Flux mort	<code>cancel_all</code> puis suspension	Après reconnexion et réconciliation
Échec d'écriture du journal	<code>Chronicle</code>	Arrêt du bot. Aucune décision non traçable n'est autorisée	Après réparation
Régime hors bornes	À $\tau = 30$ s puis chaque tick	<code>REGIME_NON_REPRESENTATIF</code> , aucune cotation sur la session, alerte, révision obligatoire de δ_{guard}	Session suivante dans les bornes

4.3 Ce que la séquence garantit

1. **Aucune décision n'est prise sans que τ soit exact.** Étape 2 avant étape 4, et `endDate` bloquant.
2. **Aucune mesure n'est calculée sur des données sales.** Étape 3 avant étape 4.
3. **Aucune économie n'est évaluée avant l'intégrité.** Étape 7 avant étape 8, et l'ordre d'évaluation de sortie met V0 et V7 avant V1, V2, V5.
4. **Aucun ordre ne part sans que l'état interne soit réconcilié.** Étape 9 avant étape 13.
5. **Aucun double ordre.** Verrou d'unicité à l'étape 9, action unique à l'étape 11, assertion à l'étape 12.
6. **Aucune décision non traçable.** Étape 16 obligatoire, échec d'écriture = arrêt.
7. **La traversée du strike passe devant tout.** Priorité 1 à l'étape 11. C'est le seul cas où le mécanisme peut devenir directionnel malgré ses gardes.

Budget de latence : la contrainte la plus dure et la moins connue. La fenêtre utile d'une quote est de **2 s**, l'annulation cible ≤ 500 ms, la détection de traversée < 300 ms. Le budget réel du bot est **non déterminable** aujourd'hui : c'est exactement ce que les 8 horodatages de l'étape 16 servent à mesurer. Tant qu'ils ne sont pas mesurés, **aucun mécanisme ne peut être déclaré atteignable.**

5. Architecture des fichiers

42 fichiers de source, 30 fichiers de test, 72 au total. Aucun fichier ne regroupe deux responsabilités différentes, et aucun fichier n'existe « au cas où ». L'arborescence générique proposée dans votre brief est adaptée : j'ai séparé `journal` de `logging` (le journal de stratégie n'est pas un logger technique), ajouté `regime`, `anchor`, `fees`, `decision` et `state`, et fusionné `indicators` dans `measure`.

5.1 Arborescence

bot/	
├── pyproject.toml	dépendances épinglées, outillage, seuils de couverture
├── README.md	thèse, limites, checklist de passage en live
├── config/	
├── constants.py	identités et constantes physiques, NON configurables
├── params.yaml	paramètres configurables, versionnés
├── params.py	chargement, validation de bornes, versionnage
└── modes.py	drapeaux de mode et leurs préconditions
├── core/	= module Kernel
├── types.py	snapshots, MeasureSet, ObservedTerms, Decision, OrderIntent
├── codes.py	25 codes de refus atomiques, 10 états de décision, 4 sous-groupes
├── units.py	Dollar, ContractPoint, MarketSecond, CaptureSecond (types disjoints)
└── tick.py	arrondi tick, prix complémentaire, côté miroir
├── journal/	= module Chronicle (RANG 1)
├── chronicle.py	écriture append-only, arrêt sur échec
├── schema.py	24 champs + 8 horodatages + sous-groupe + code
└── health.py	agrégats hors ligne, lag_signé, taux de refus par code
├── clock/	= module Chronos
└── chronos.py	τ_0 , τ , phase, paliers, $w(\tau)$, τ_{lock}
├── ingest/	= module Intake
├── intake.py	union des événements, âge, trou, disponibilité
├── dedup.py	doublons et événements désordonnés
└── streams.py	adaptateurs oracle, carnet, spot, trades, officiel
├── anchor/	= module Anchor
└── anchor.py	K, k_source, m*, côté miroir, traversée
├── measure/	
├── oracle.py	= module Oracle : $\sigma_{30}(A)$, σ_T , λ , z , $\Phi(z)$, X_{60} , P, v, TWAP30
└── garnet.py	= module Garnet : ask, bid, spread, e, méd _{3s} , q95, invariants, δ
├── fees/	= module Fees
└── fees.py	fee_sizing, fee_settlement, fee_exit, garde fee_bps, bande morte
└── microstructure/	= module Prism

└─ prism.py	spread borné, anti-couteau, stabilité, vitesse, profondeur	
└─ regime/	= module Regime	
└─ regime.py	ratio sur 20 sessions, volatilité annualisée implicite	
└─ validation/	= module Warden	
└─ warden.py	pile de vetos fail-closed, arrêt au premier faux	
└─ strategy/		
└─ signal.py	= module Signal : porte d'entrée, prix limite, taille	
└─ exit.py	= module Exit : V0 à V8, verrou terminal, suspensions	
└─ mechanisms.py	M1, M2, M3 déclarés avec leurs préconditions et leur ordre	
└─ risk/	= module Risk	
└─ risk.py	N, PARTS_MIN, capacité, budget de mesure, kill-switch	
└─ decision/	= module Arbiter	
└─ arbiter.py	priorités, idempotence, 7 assertions, validation finale	
└─ execution/	= module Executor	
└─ executor.py	verrou d'unicité, post_only, validité 2 s, annulation	
└─ lifecycle.py	machine à états d'un ordre, 8 horodatages	
└─ broker.py	client API, SEUL point réseau sortant du projet	
└─ state/	= module Positions	
└─ positions.py	position, ordres en vol, réconciliation, sous-groupe, PnL	
└─ dry_run/	= module Harness	
└─ harness.py	rejeu déterministe, horloge simulée	
└─ mirror.py	génération de jeux de données miroir	
└─ fill_sim.py	simulateur : contact BBO et fill ÉTIQUETÉS DIFFÉREMMENT	
└─ sweep.py	balayage de configurations et de δ	
└─ report.py	PnL par sous-groupe, Clopper-Pearson, rapport PAR SESSION	
└─ gate/	= module Gate	
└─ gate.py	11 critères de passage en live, critères de retrait	
└─ main/		
└─ run_replay.py	rejeu hors ligne sur sessions enregistrées	
└─ run_dry.py	dry run temps réel, aucun ordre envoyé	
└─ run_live.py	live, refuse de démarrer si Gate n'est pas vert	
└─ tests/		
└─ static/		
└─ test_no_sentinels.py	aucune sentinelle numérique	(R21)
└─ test_no_frozen_sigma.py	σ_{30} jamais constante	(R14)
└─ test_single_sigma_impl.py	une seule implémentation de σ_{30}	(R15)
└─ test_fee_isolation.py	pas de réutilisation croisée	(R41)
└─ test_no_directional_terms.py	p, D signé, lag, pente, Φ_{30} non lus	(R13, 24, 25, 11)
└─ test_no_deleted_constants.py	P_max, 0.92, CAPITAL_DEMI absents	(R20, 30, 47)
└─ test_no_magic_values.py	toute littérale vient de config	
└─ unit/	19 fichiers, un par module	
└─ test_<module>.py		
└─ integration/		
└─ test_full_tick.py	les 17 étapes de bout en bout	
└─ test_degraded_paths.py	les 19 situations dégradées	
└─ test_no_double_order.py	rafale de 1 000 événements	

		test_mirror_symmetry.py	non-directionnalité opérationnelle
		test_corpus_regression.py	reproduit 0 entrée, lag ≤ 0, $\sigma_{30} \approx 7,20$ \$
		test_registry_coverage.py	CHACQUE entrée du registre a ≥ 1 test
		fixtures/	
		sessions/	CSV de sessions réelles
		mirrored/	jeux miroir générés
		synthetic/	cas limites : spread croisé, invariant rompu, traversée de strike, krach de 0,279 s, trou de 7 s, événements désordonnés

5.2 Rôle et dépendances par fichier

Fichier	Rôle	Fonctions principales	Dépend de
config/constants.py	Constantes et identités non configurables : TICK = 0,01 , K_LAMBDA = 0,5513 , T = 300 , SIGMA_WINDOW = 30 , PARTS_MIN = 5	— (déclarations)	aucune
config/params.yaml	Paramètres configurables avec leur statut : z_min = 1,0 (PROVISOIRE), δ_{guard} par palier, D_max , seuils de fraîcheur, τ_{min} , τ_{max} , τ_{lock} , C , bornes de régime, planchers et plafonds de quantiles	—	aucune
config/params.py	Chargement, validation de bornes, versionnage. Refuse de démarrer si un paramètre est hors bornes ou non versioné	load() , validate() , version()	constants
config/modes.py	Drapeaux et leurs préconditions : PASSIVE_ONLY / TAKER_ASSUMED (R36), SETTLE_ONLY / EARLY_EXIT (R42), M3_INSTRUMENTATION / M1_LIVE	may_enable(mode)	params
core/types.py	Toutes les structures immuables	définitions de types	units

<code>core/codes.py</code>	25 codes atomiques, 10 états, 4 sous-groupes	<code>Subgroup.of()</code>	aucune
<code>core/units.py</code>	Types d'unités disjoints . Le mélange des deux horloges échoue au typage	opérateurs restreints	aucune
<code>core/tick.py</code>	Arrondi, prix complémentaire, côté miroir	<code>round_down_to_tick</code> , <code>complement_price</code> , <code>mirror_side</code>	<code>units</code> , <code>constants</code>
<code>journal/schema.py</code>	Schéma figé du journal. Toute évolution est versionnée	<code>Record</code> , <code>serialize</code>	<code>core/*</code>
<code>journal/chronicle.py</code>	Écriture append-only. Échec d'écriture = arrêt du bot	<code>log_decision</code> , <code>log_refusal</code> , <code>log_order_lifecycle</code>	<code>schema</code>
<code>journal/health.py</code>	Agrégats hors ligne. Consomme <code>lag_signé</code> , <code>p</code> , <code>D</code> , <code>momentum</code> , que la décision n'a pas le droit de lire	<code>health_report</code> , <code>refusal_distribution</code>	<code>chronicle</code>
<code>clock/chronos.py</code>	Seule source de temps	<code>open_session</code> , <code>tau_now</code> , <code>phase</code> , <code>w</code> , <code>is_locked</code>	<code>core/*</code> , <code>journal</code>
<code>ingest/streams.py</code>	Adaptateurs par flux. Traduit les charges brutes en types du <code>Kernel</code>	<code>OracleStream</code> , <code>BookStream</code> , <code>SpotStream</code> , <code>TradeStream</code> , <code>OfficialStream</code>	<code>core/*</code>
<code>ingest/dedup.py</code>	Doublons et désordre	<code>is_duplicate</code> , <code>is_out_of_order</code>	<code>core/*</code>
<code>ingest/intake.py</code>	Union des événements, 4 Hz minimum, âge, trou, disponibilité	<code>next_event</code> , <code>age</code> , <code>gap</code> , <code>anomaly_counters</code>	<code>streams</code> , <code>dedup</code> , <code>clock</code> , <code>journal</code>
<code>anchor/anchor.py</code>	Strike certifié, <code>m*</code> , côté miroir, traversée	<code>resolve_strike</code> , <code>m_star</code> , <code>anchored_side</code> , <code>crossed_since</code>	<code>ingest</code> , <code>clock</code>
<code>measure/oracle.py</code>	Une seule implémentation de σ_{30} (estimateur A), toutes les grandeurs oracle	<code>sigma_30</code> , <code>sigma_T</code> , <code>z_of</code> , <code>anchored_price</code> , <code>x60</code> , <code>persistence</code> , <code>velocity</code> , <code>twap_30</code>	<code>anchor</code> , <code>clock</code> , <code>ingest</code>

<code>measure/garnet.py</code>	Tous les termes carnet, en valeur absolue	<code>side_prices</code> , <code>gap</code> , <code>q95_jump</code> , <code>q95_drop</code> , <code>invariant_error</code> , <code>basis_of</code> , <code>decote</code>	<code>oracle</code> , <code>fees</code> , <code>ingest</code>
<code>fees/fees.py</code>	Trois fonctions de frais non interchangeables	<code>fee_sizing</code> , <code>fee_settlement</code> , <code>fee_exit</code> , <code>fee_guard_ok</code> , <code>dead_band</code>	<code>core/*</code>
<code>microstructure/prism.py</code>	Cinq filtres, chacun avec valeur mesurée et seuil appliqué	<code>spread_ok</code> , <code>no_knife</code> , <code>ask_stable</code> , <code>velocity_ok</code> , <code>depth_ok</code>	<code>garnet</code> , <code>oracle</code>
<code>regime/regime.py</code>	Garde d'universalité de la volatilité	<code>regime_ratio</code> , <code>regime_ok</code> , <code>implied_annual_vol</code>	<code>oracle</code> , <code>journal</code>
<code>validation/warden.py</code>	Pile de vetos, ordre déterministe, arrêt au premier faux	<code>evaluate_all</code> , <code>integrity_ok</code> , <code>freshness_ok</code> , <code>session_ok</code> , <code>state_ok</code>	<code>anchor</code> , <code>oracle</code> , <code>garnet</code> , <code>prism</code> , <code>regime</code> , <code>state</code>
<code>strategy/mechanisms.py</code>	M1, M2, M3 déclarés avec préconditions et ordre imposé (M2, puis M3, puis M1)	<code>M2</code> , <code>M3</code> , <code>M1</code> , <code>active_mechanisms(modes)</code>	<code>modes</code> , <code>warden</code>
<code>strategy/signal.py</code>	Porte d'entrée, prix limite selon le mode, taille	<code>evaluate</code> , <code>limit_price</code> , <code>eligible</code>	<code>warden</code> , <code>risk</code> , <code>mechanisms</code>
<code>strategy/exit.py</code>	V0 à V8 dans l'ordre imposé, verrou terminal, suspensions	<code>decide</code> , <code>v0</code> ... <code>v8</code>	<code>warden</code> , <code>fees</code> , <code>state</code>
<code>risk/risk.py</code>	Sizing unique, capacité, budget, kill-switch, détection d'adaptation	<code>size</code> , <code>capacity_ok</code> , <code>measurement_budget_left</code> , <code>detect_adaptation</code>	<code>garnet</code> , <code>fees</code> , <code>state</code>
<code>decision/arbiter.py</code>	Une action par événement, priorités, 7 assertions	<code>decide</code> , <code>assert_consistency</code> , <code>is_duplicate</code>	<code>signal</code> , <code>exit</code> , <code>risk</code> , <code>state</code> , <code>journal</code>
<code>execution/lifecycle.py</code>	Machine à états d'un ordre, 8 horodatages	<code>OrderLifecycle</code> , transitions	<code>core/*</code>
<code>execution/broker.py</code>	Seul point réseau sortant du projet. Aucun autre fichier n'ouvre de socket	<code>place</code> , <code>cancel</code> , <code>fetch_state</code> , <code>fetch_fee_bps</code>	<code>core/*</code>
<code>execution/executor.py</code>	Verrou d'unicité, <code>post_only</code> , validité 2 s,	<code>submit</code> , <code>cancel</code> , <code>cancel_all</code> , <code>on_*</code> ,	<code>arbiter</code> , <code>broker</code> , <code>lifecycle</code> , <code>state</code> ,

	annulation ≤ 500 ms, un seul réessai	<code>in_flight_count</code>	<code>journal</code>
<code>state/positions.py</code>	État vérité, réconciliation, sous-groupe, PnL réel et contrefactuel	<code>snapshot</code> , <code>apply</code> , <code>reconcile</code> , <code>settle</code> , <code>subgroup</code>	<code>core/*</code> , <code>journal</code> , <code>fees</code>
<code>dry_run/fill_sim.py</code>	Simulateur d'appariement. Contact BBO et fill sont deux étiquettes distinctes , jamais confondues	<code>simulate</code> , <code>bbo_contact</code> , <code>fill_at_ask</code>	<code>core/*</code>
<code>dry_run/mirror.py</code>	Génération de jeux miroir : l'outil du test de non-directionnalité	<code>mirror(session)</code>	<code>core/*</code>
<code>dry_run/harness.py</code>	Rejeu déterministe, horloge simulée	<code>replay</code>	tous
<code>dry_run/sweep.py</code>	Balayage de configurations et de δ	<code>sweep</code>	<code>harness</code>
<code>dry_run/report.py</code>	Rapport par session , PnL par sous-groupe, Clopper-Pearson, table de fill par phase et par δ	<code>subgroup_pnl</code> , <code>fill_rate_table</code> , <code>clopper_pearson</code>	<code>harness</code>
<code>gate/gate.py</code>	11 critères, verdict binaire, critères de retrait	<code>evaluate_readiness</code> , <code>may_go_live</code> , <code>withdrawal_triggered</code>	<code>report</code> , <code>health</code>
<code>main/run_replay.py</code>	Rejeu hors ligne. Aucun réseau	<code>main()</code>	<code>harness</code>
<code>main/run_dry.py</code>	Dry run temps réel. Flux réels, aucun ordre envoyé , journalisation complète	<code>main()</code>	tous sauf <code>broker.place</code>
<code>main/run_live.py</code>	Live. Refuse de démarrer si <code>Gate.may_go_live()</code> est faux	<code>main()</code>	tous + <code>gate</code>

5.3 Fichiers par rôle d'exécution

Besoin	Fichiers requis
--------	-----------------

Configuration	<code>config/constants.py</code> , <code>config/params.yaml</code> , <code>config/params.py</code> , <code>config/modes.py</code>
Journalisation	<code>journal/chronicle.py</code> , <code>journal/schema.py</code> , <code>journal/health.py</code>
Tests	<code>tests/static/</code> (7), <code>tests/unit/</code> (19), <code>tests/integration/</code> (6), <code>tests/fixtures/</code>
Simulation	<code>dry_run/harness.py</code> , <code>fill_sim.py</code> , <code>mirror.py</code> , <code>sweep.py</code> , <code>report.py</code>
Dry run	Tous les fichiers de source sauf l'appel à <code>broker.place()</code> , plus <code>main/run_dry.py</code> . Le dry run n'est pas un sous-ensemble du code : c'est le même code avec un broker inerte, sinon le dry run ne prouve rien
Passage en live	<code>gate/gate.py</code> , <code>main/run_live.py</code> , plus les 11 champs officiels câblés dans <code>ingest/streams.py</code> et <code>execution/broker.py</code>



Deux invariants d'arborescence, vérifiés par test statique. Premier : `execution/broker.py` est le **seul** fichier autorisé à ouvrir une connexion sortante. Second : `journal/` est le **seul** fichier autorisé à écrire sur disque. Tout le reste est pur, donc rejouable et testable sans I/O.

6. Plan de test et plan de codage

6.1 Plan de test

Quatre niveaux, et un cinquième qui n'est pas un niveau mais une obligation.

Niveau	Objet	Critère de passage
Statique (7 fichiers)	Interdictions structurelles vérifiées sur la base de code elle-même : pas de sentinelle, pas de σ_{30} gelée, une seule implémentation de σ_{30} , pas de réutilisation croisée des frais, pas de lecture de grandeur directionnelle, pas de constante supprimée, pas de valeur magique	Zéro violation. Ces tests échouent à la présence d'un motif, pas à un résultat numérique. Ce sont eux qui empêchent les erreurs de l'étape 3 de revenir par la porte de derrière
Unitaire (19 fichiers)	Un fichier par module, avec les tests énumérés au point 11 de chaque spécification de la page 3	Couverture ≥ 95 % sur les modules purs, 100 % sur <code>Kernel</code> et <code>Fees</code>
Propriétés (inclus dans l'unitaire)	Invariants vérifiés sur données générées : <code>complement_price</code> est une involution ; <code>δ</code> est monotone décroissante par paliers ; <code>$e \geq 0$</code> toujours ; <code>σ_T</code> strictement décroissante ; <code>$\sigma_T(\tau) = \sigma_T(0)/\sqrt{5}$</code> à $\tau = 240$	Zéro contre-exemple sur 10 000 tirages
Intégration (6 fichiers)	Les 17 étapes de bout en bout ; les 19 situations dégradées ; absence de double ordre sous rafale ; symétrie miroir ; régression sur le corpus ; couverture du registre	Voir ci-dessous

Obligation	<code>test_registry_coverage.py</code> parcourt le registre des 50 entrées et vérifie que chacune est reliée à au moins un test nommé et existant	50/50. Une entrée sans test fait échouer la suite complète. C'est la traduction mécanique de votre règle : aucune correction n'est intégrée si elle n'est pas reliée à un test vérifiable
-------------------	---	---

Les trois tests d'intégration qui comptent vraiment

1. `test_corpus_regression.py` : **le harnais avant la stratégie**. Le rejeu du corpus sous la configuration de l'étape 3 doit reproduire trois chiffres : **0 entrée**, **lag médian ≤ 0 s**, **σ_{30} médian $\approx 7,20$ \$** (estimateur A). S'il ne les reproduit pas, le harnais est faux et aucun résultat de stratégie ne vaut rien. C'est le premier test à écrire après le journal.
2. `test_mirror_symmetry.py` : **le test de non-directionnalité opérationnel**. Sur un jeu de données miroir généré (oracle réfléchi autour de K), le bot doit produire des décisions rigoureusement miroir : `|m*|`, `z`, `$\sigma_T$` , `$\delta$` , `e`, `N` identiques, côté inversé, prix complémentaire. **Une asymétrie révèle un pari caché**. C'est le seul test qui pourra un jour prouver que le mécanisme n'est pas directionnel, et c'est le test que le corpus ne permettait pas de faire (3 sessions, toutes DOWN).
3. `test_no_double_order.py` : **la garde binaire**. Rafale de 1 000 événements en 1 s, avec doublons, désordre, acks retardés, fills partiels et déconnexions. Zéro double ordre, zéro état incohérent, zéro fill sur ordre périmé. **Un seul échec suffit à invalider**.

Ce que les tests ne peuvent pas prouver

 Aucun test ne peut prouver que la stratégie est rentable. Avec 3 issues indépendantes, un marqueur à 3/3 a un IC95 de Clopper-Pearson de **[29,2 % ; 100 %]** : un taux réel de 30 % est statistiquement compatible avec les données. Il faut **29 sessions consécutives sans faute** pour rejeter $H_0(p \leq 0,90)$. Tous les taux de 100 % du corpus sont **descriptifs, pas prédictifs**. Ce que les tests prouvent, c'est que le code fait ce que la spécification dit, et que la spécification ne contient pas de pari caché. Rien de plus.

6.2 Plan de codage

Quinze étapes, dans l'ordre imposé par les dépendances. **Aucune étape ne commence avant que la précédente soit verte**.

Étape 0 (rang 1 absolu) : la journalisation

Fichiers	<code>core/units.py</code> , <code>core/codes.py</code> , <code>core/types.py</code> , <code>core/tick.py</code> , <code>journal/schema.py</code> ,
-----------------	---

	<code>journal/chronicle.py</code> , <code>config/*</code>
Fonctions	<code>round_down_to_tick</code> , <code>complement_price</code> , <code>mirror_side</code> , <code>Subgroup.of</code> , <code>log_decision</code> , <code>log_refusal</code> , <code>log_order_lifecycle</code> , <code>params.load/validate/version</code>
Prérequis	Aucun. Aucune ligne de code de trading n'est écrite avant celle-ci.
Tests	T-K1 à T-K4, T-CH1 à T-CH5, <code>test_no_sentinels.py</code> , <code>test_no_magic_values.py</code>
Erreurs à éviter	Sentinelles numériques (le <i>fail-open</i> de l'étape 1). Horloges mélangées. Un code de refus non atomique (l'étape 3 en avait 14 pour 22 tests). Littérales dans le code métier.
Validation	100 % des décisions et des refus journalisés avec code atomique unique. Zéro ligne écrite après confirmation.
Registre couvert	R07, R21, R33 (partiel)

Étape 1 : structures de données communes et temps

Fichiers	<code>clock/chronos.py</code> , <code>fees/fees.py</code>
Fonctions	<code>open_session</code> , <code>tau_now</code> , <code>phase</code> , <code>w</code> , <code>is_in_action_window</code> , <code>is_locked</code> , <code>fee_sizing</code> , <code>fee_settlement</code> , <code>fee_exit</code> , <code>fee_guard_ok</code> , <code>dead_band</code>
Prérequis	Étape 0 verte
Tests	T-C1 à T-C5, T-F1 à T-F4, <code>test_fee_isolation.py</code>

Erreurs à éviter	<code>time()</code> comme origine. <code>t_ms</code> relatif pris pour absolu. Réutilisation croisée des trois modèles de frais.
Validation	Refus de session sans <code>endDate</code> . Les trois fonctions de frais sont prouvées distinctes.
Registre couvert	R02, R06, R12, R41

Étape 2 : validation et normalisation des données

Fichiers	<code>ingest/streams.py</code> , <code>ingest/dedup.py</code> , <code>ingest/intake.py</code>
Fonctions	<code>next_event</code> , <code>age</code> , <code>gap</code> , <code>is_duplicate</code> , <code>is_out_of_order</code> , <code>anomaly_counters</code>
Prérequis	Étape 1 verte
Tests	T-I1 à T-I6
Erreurs à éviter	Évaluation à 1 Hz (98 % des mises à jour invisibles). Recalcul rétroactif sur événement désordonné. Doublon qui avance un compteur de persistance. Usage de <code>ts_src</code> comme heure.
Validation	Cadence effective ≥ 4 Hz mesurée et publiée. Le krach de 0,279 s est vu.
Registre couvert	R10, R11, R31

Étape 3 : mesure des données brutes, `Oracle` TWAP/FIX

Fichiers	<code>anchor/anchor.py</code> , <code>measure/oracle.py</code>
Fonctions	<code>resolve_strike</code> , <code>m_star</code> , <code>anchored_side</code> , <code>crossed_since</code> , <code>sigma_30</code> ,

	<code>sigma_T</code> , <code>lambda_of</code> , <code>twap_30</code>
Prérequis	Étape 2 verte
Tests	T-A1 à T-A4, T-O1 à T-O3, T-O7, T-O8, <code>test_no_frozen_sigma.py</code> , <code>test_single_sigma_impl.py</code>
Erreurs à éviter	Toute la stérilité de l'étape 3 est ici. σ_{30} gelée à 26,50 \$ (facteur $\times 3,68$, 0 entrée sur tout le corpus). Deux estimateurs coexistants. Un proxy de strike (12,25 \$ d'écart inversent un règlement). Division par zéro à $\tau \rightarrow 300$.
Validation	Une seule implémentation de σ_{30} dans la base de code. Médiane sur 500 sessions publiée avec P10/P90, sortie de [0,5 ; 40] sur $\leq 5\%$ des sessions. <code>K</code> officiel sur $\geq 99\%$ des sessions.
Registre couvert	R01, R14, R15, R29

Étape 4 : calcul des termes observés

Fichiers	<code>measure/oracle.py</code> (suite), <code>measure/garnet.py</code>
Fonctions	<code>z_of</code> , <code>anchored_price</code> , <code>x60</code> , <code>persistence</code> , <code>velocity</code> , <code>side_prices</code> , <code>gap</code> , <code>rolling_median</code> , <code>q95_jump</code> , <code>q95_drop</code> , <code>invariant_error</code> , <code>basis_of</code> , <code>decote</code>
Prérequis	Étape 3 verte
Tests	T-O4 à T-O6, T-GA1 à T-GA6, <code>test_no_directional_terms.py</code>

Erreurs à éviter	D signé (anti-corrélé au résultat). k_λ traité en hyperparamètre. p de Platt traité en probabilité. Quantiles codés en constantes. Basis employé en régression prédictive. Consommation de la régression $e(\tau)$ contestée.
Validation	$e \geq 0$ toujours. Prix NON dérivé = prix NON observé à 1 tick sur 1 025 lignes. $\delta \geq \delta_{\min}$ et multiple du tick.
Registre couvert	R24, R25, R27, R28, R34, R37, R39

Étape 5 : indicateurs et mesures dérivées, plus la garde de régime

Fichiers	<code>regime/regime.py</code>
Fonctions	<code>regime_ratio</code> , <code>regime_ok</code> , <code>implied_annual_vol</code> , <code>record_session</code>
Prérequis	Étape 4 verte, plus un historique de 20 sessions
Tests	T-RG1 à T-RG3
Erreurs à éviter	Appliquer un calibrage de régime calme à 6,9 % de volatilité annualisée à un régime BTC normal à 40 à 60 %. Démarrer sans historique.
Validation	Volatilité annualisée implicite publiée par session. Historique < 20 sessions $\Rightarrow$ refus.
Registre couvert	R35

Étape 6 : filtres de microstructure

Fichiers	<code>microstructure/prism.py</code>
Fonctions	<code>spread_ok</code> , <code>no_knife</code> , <code>ask_stable</code> , <code>velocity_ok</code> ,

	<code>depth_ok</code>
Prérequis	Étape 5 verte
Tests	T-P1 à T-P6
Erreurs à éviter	Spread sans borne basse (2 carnets croisés passaient). Seuils absolus 10 à 20× trop serrés. Garde de pente unilatérale. Simuler une profondeur absente.
Validation	Taux de refus par test entre 5 % et 30 %. Aucun test à 0 % ni à 100 %.
Registre couvert	R03, R17, R18, R21, R23, R45

Étape 7 : suivi des positions (avancé pour briser le cycle)

Fichiers	<code>state/positions.py</code>
Fonctions	<code>snapshot</code> , <code>apply</code> , <code>reconcile</code> , <code>settle</code> , <code>subgroup</code>
Prérequis	Étape 0 verte. Avancé ici parce que <code>Warden</code> en dépend en lecture seule : c'est ainsi que le cycle <code>Warden ↔ Positions</code> est brisé.
Tests	T-PO1 à T-PO4
Erreurs à éviter	Position fantôme tolérée. PnL calculé avec <code>fee_sizing</code> . Sous-groupe non renseigné.
Validation	Réconciliation 100 %, zéro position fantôme. Sous-groupe sur 100 % des événements.
Registre couvert	R04, R09

Étape 8 : blocs de validité

Fichiers	<code>validation/warden.py</code>
-----------------	-----------------------------------

Fonctions	<code>evaluate_all</code> , <code>integrity_ok</code> , <code>freshness_ok</code> , <code>session_ok</code> , <code>state_ok</code>
Prérequis	Étapes 6 et 7 vertes
Tests	T-W1 à T-W7, <code>test_no_deleted_constants.py</code>
Erreurs à éviter	Le piège principal : reconstruire un filtre stérilisant. Un test qui refuse 100 % est faux par construction. <code>D_max</code> en condition d'entrée (R19, R38). <code>P_max</code> et ses trois constantes. Basis unilatéral. Deux codes vrais simultanément.
Validation	Zéro cotation sur données incohérentes. Taux de refus par code cohérent avec le corpus (invariant $\approx 0,2$ %, spread ≈ 14 %). PnL 0,00 \$ dans chaque sous-groupe en mode veto.
Registre couvert	R05, R16, R19, R20, R22, R32, R33, R38, R44, R50

Étape 9 : gestion du risque

Fichiers	<code>risk/risk.py</code>
Fonctions	<code>size</code> , <code>capacity_ok</code> , <code>measurement_budget_left</code> , <code>kill_switch_state</code> , <code>detect_adaptation</code>
Prérequis	Étape 8 verte
Tests	T-R1 à T-R6
Erreurs à éviter	Deux formules de sizing. Branche demi-taille morte. Ignorer que 10 % des événements portent 63 % du notionnel et que 3 des 5 pics de gros flux tombent après 240 s,

	donc dans la fenêtre d'action.
Validation	Une seule formule. Budget de mesure borné à 400 \$ sur 100 sessions. Ratio PnL / capital > +1,0 % par événement rempli, dans chaque sous-groupe.
Registre couvert	R30, R40

Étape 10 : moteur de signal

Fichiers	<code>strategy/mechanisms.py</code> , <code>strategy/signal.py</code>
Fonctions	<code>M2</code> , <code>M3</code> , <code>M1</code> , <code>active_mechanisms</code> , <code>evaluate</code> , <code>limit_price</code> , <code>eligible</code>
Prérequis	Étape 9 verte. <code>**R36</code> tranché.** Cette étape ne peut pas commencer sans votre décision sur la règle de prix.
Tests	T-S1 à T-S6
Erreurs à éviter	Appeler « passif » un ordre qui traverse le spread. Fenêtre [60;240] au lieu de [180;270]. Consommer <code>p</code> , <code>D</code> signé, <code>lag</code> , la pente. <code>δ</code> sous son plancher. Activer <code>TAKER_ASSUMED</code> sans calibration.
Validation	≥ 0,5 événement éligible par session. Répartition par couleur publiée. En <code>PASSIVE_ONLY</code> , aucun prix limite au-dessus du bid.
Registre couvert	R13, R26, R36, R37

Étape 11 : moteur de sortie

Fichiers	<code>strategy/exit.py</code>
----------	-------------------------------

Fonctions	<code>decide</code> , <code>v0</code> à <code>v8</code>
Prérequis	Étape 10 verte. **R42 et R48 tranchés.**
Tests	T-X1 à T-X8
Erreurs à éviter	<code>HOLD</code> silencieux quand la sortie était impossible. Utiliser l'ask pour déclarer une vente au bid. Oublier <code>q = 1 - p</code> sur une position NO. Coder 0,92 en dur. Verrou terminal à 240 s. V4 en déclencheur autonome. Empiler 9 conditions par-dessus 20 et recréer la stérilité.
Validation	Les 7 contradictions interdites lèvent. V1 et V5 inertes tant que la calibration manque. Sous <code>EARLY_EXIT</code> , comparaison obligatoire contre <code>HOLD_TO_SETTLEMENT</code> sur $\geq$ 29 sessions indépendantes réparties UP et DOWN.
Registre couvert	R42, R43, R44, R45, R46, R47, R48

Étape 12 : module de décision

Fichiers	<code>decision/arbitrator.py</code>
Fonctions	<code>decide</code> , <code>assert_consistency</code> , <code>is_duplicate</code>
Prérequis	Étape 11 verte
Tests	T-AR1 à T-AR5
Erreurs à éviter	Deux actions par événement. Priorité de la traversée de strike non absolue. Entrée avec position ouverte. Traiter entrée et sortie comme

	deux filtres indépendants plutôt qu'un double arrêt.
Validation	Zéro état incohérent sur un rejeu de 500 sessions. 100 % des actions traçables à une raison unique.
Registre couvert	R32 (priorité), R04 (verrou)

Étape 13 : module d'exécution

Fichiers	<code>execution/lifecycle.py</code> , <code>execution/broker.py</code> , <code>execution/executor.py</code>
Fonctions	<code>submit</code> , <code>cancel</code> , <code>cancel_all</code> , <code>on_ack</code> , <code>on_fill</code> , <code>on_partial</code> , <code>on_reject</code> , <code>in_flight_count</code>
Prérequis	Étape 12 verte
Tests	T-E1 à T-E6, <code>test_no_double_order.py</code>
Erreurs à éviter	Modification en place au lieu d'annulation puis recotation. <code>post_only</code> oublié. Compter un gain avant confirmation de fill. Boucle de réémission sur rejet. Traiter un rejet <code>post_only</code> comme une erreur.
Validation	Les 8 horodatages présents et croissants. Délai d'annulation ≤ 500 ms au P90. Zéro fill sur ordre périmé : garde binaire, un seul suffit à invalider.
Registre couvert	R07, R08, R32, R36

Étape 14 : mode dry run et instrumentation M3

Fichiers	<code>dry_run/fill_sim.py</code> , <code>dry_run/mirror.py</code> , <code>dry_run/harness.py</code> ,
----------	---

	dry_run/sweep.py , dry_run/report.py , main/run_replay.py , main/run_dry.py
Fonctions	replay , mirror , simulate , bbo_contact , fill_at_ask , sweep , subgroup_pnl , fill_rate_table , clopper_pearson
Prérequis	Étape 13 verte
Tests	T-H1 à T-H6, test_corpus_regression.py , test_mirror_symmetry.py , test_full_tick.py , test_degraded_paths.py
Erreurs à éviter	Compter un contact BBO comme un fill. Supposer une profondeur inconnue. Publier un agrégat sans ses 4 sous-groupes. Rapport par seconde au lieu de par session. Rejeu non déterministe.
Validation	Rejeu reproduisant 0 entrée sous la configuration de l'étape 3, lag médian ≤ 0 , $\sigma_{30} \approx 7,20$ \$. Symétrie miroir exacte. Puis M3 sur ≥ 100 sessions : taux de fill réel par phase et par δ , table de décote reconstruite avec des fills, PnL par sous-groupe renseigné pour la première fois.
Registre couvert	R08, R09, R16, R36 (mesure décisive)

Étape 15 : contrôles de passage en live

Fichiers	gate/gate.py , main/run_live.py , journal/health.py
----------	--

Fonctions	<code>evaluate_readiness</code> , <code>may_go_live</code> , <code>may_enable</code> , <code>withdrawal_triggered</code> , <code>health_report</code>
Prérequis	Étape 14 verte, plus ≥ 100 sessions de M3
Tests	T-G1 à T-G5, <code>test_registry_coverage.py</code>
Erreurs à éviter	Traiter un critère non évaluable comme neutre au lieu de rouge. Retenir la valeur qui maximise l'agrégat. Réajuster au lieu de retirer. Réintroduire <code>p</code> avec un seul sous-groupe en échec.
Validation	Les 11 critères verts, journalisés et versionnés. PnL strictement positif dans CHACUN des 4 sous-groupes sur ≥ 200 sessions, avec ≥ 100 sessions par sous-groupe.
Registre couvert	R24, R35, plus la vérification que les 50 entrées sont couvertes

6.3 Ordre des mécanismes, non négociable

M2 (intégrité) aucun ordre PnL 0,00 \$	→	M3 (instrumentation, 5 parts) ordres minimaux espérance ≈ 0 , budget ≤ 400 \$	→	M1 (cotation, taille réelle N) ordres réels PnL à démontrer
--	---	--	---	---

M3 avant M1 n'est pas une précaution : **c'est le seul mécanisme qui transforme les onze champs manquants en données**. Sans lui, la table de décote restera pour toujours une table de contacts BBO, et aucun PnL par sous-groupe ne sera jamais calculable. Et c'est aussi lui qui tranchera empiriquement R36 : si les ordres reviennent systématiquement `maker` , le mécanisme est passif ; s'ils reviennent `taker` , la porte produit bien des ordres agressifs et l'économie du livrable est à refaire.

6.4 Couverture du registre par le plan de codage

Étape	Entrées du registre couvertes	Nombre
0. Journalisation	R07, R21, R33	3
1. Temps et frais	R02, R06, R12, R41	4
2. Ingestion	R10, R11, R31	3
3. Oracle et ancrage	R01, R14, R15, R29	4
4. Termes observés	R24, R25, R27, R28, R34, R37, R39	7
5. Régime	R35	1
6. Microstructure	R03, R17, R18, R21, R23, R45	6
7. Positions	R04, R09	2
8. Validité	R05, R16, R19, R20, R22, R32, R33, R38, R44, R50	10
9. Risque	R30, R40	2
10. Signal	R13, R26, R36, R37	4
11. Sortie	R42, R43, R44, R45, R46, R47, R48	7
12. Décision	R04, R32	2
13. Exécution	R07, R08, R32, R36	4
14. Dry run et M3	R08, R09, R16, R36	4
15. Passage en live	R24, R35	2
Total	49 entrées sur 50 couvertes par au moins une étape et un test	49/50
Non couverte	R49 (documents manquants) : ce n'est pas une correction de code, c'est une dépendance sur vous	1

7. Points nécessitant votre validation

 Cinq décisions bloquent le codage. Sept demandes sont importantes mais ne bloquent que des étapes tardives. Rien ne sera codé avant votre accord.

A. Les cinq décisions bloquantes

D1. Règle de prix : le mécanisme est-il passif ou agressif ? (R36)

Pourquoi c'est nécessaire. La porte de l'étape 4 garantit $p_{\text{ancré}} - \delta > \text{ask}$, donc un ordre limite à ce prix traverse le spread et s'exécute en taker. Le PnL annoncé de +0,075 \$ par événement suppose une capture de spread par quote passive, qui n'arrive jamais. L'économie réelle serait de **+1,18 \$ ou -4,80 \$** à ask = 0,80, avec un point mort $\approx$ ask, soit 80 à 96 % de réussite directionnelle exigée. Je ne peux écrire ni `Signal` ni `Executor` sans savoir laquelle des deux natures coder.

Option	Conséquence
A. `PASSIVE_ONLY` <code>prix_limite = min(arrondi_tick_bas(p_ancré - δ), bid_côté), post_only</code> obligatoire	Tenue de marché. Cohérent avec toute la démonstration de l'étape 4. <code>e</code> devient une condition d'éligibilité et non un prix. Le taux de fill réel devient la mesure critique
B. `TAKER_ASSUMED` <code>prix_limite = ask_côté</code>	Achat de valeur directionnel. Exige $\Phi(z)$ calibré par sous-groupe avant tout ordre , ce que l'étape 4 interdit aujourd'hui. Et rend le moteur de sortie de l'étape 5 obligatoire

Ma recommandation : A, sans hésitation. L'option B parie que $\Phi(z)$ bat le carnet, alors que le corpus mesure $R^2(m^*, \text{mid}) = 0,904$ et $R^2(p_{\text{brut}}, \text{ask}) = 0,956$: le carnet réplique déjà l'oracle, il ne sait rien de moins. Et le Brier par sous-groupe réfute déjà la calibration. Sous B, le bot redevient exactement ce que l'étape 4 a passé 9 988 lignes à démonter.

D2. Politique de sortie : règlement unique ou sortie anticipée ? (R42)

Pourquoi c'est nécessaire. L'étape 4 écrit « sortie unique, au règlement, pas de sortie intermédiaire ». L'étape 5 construit un moteur complet de sortie anticipée. Les deux documents ne décrivent pas le même bot, et la machine à états ne peut pas être figée sans trancher.

Ma recommandation : `SETTLE_ONLY` si vous choisissez A en D1, `EARLY_EXIT` si vous choisissez B. Ce n'est pas une préférence, c'est une conséquence : une quote passive vend de l'attente, donc en sortir tôt détruit la thèse ; une position directionnelle remplie au ask a besoin de stops. Dans les deux cas, `SUSPEND_DATA` et `SUSPEND_LIQUIDITY` restent actifs.

D3. Le coupe-circuit `D_max` doit-il rester dans la conjonction d'entrée ? (R38)

Pourquoi c'est nécessaire. La porte contient `e ≤ 0,114`, alors que l'écart médian mesuré vaut **0,4642 pt** en [240;270[. C'est exactement l'erreur que la section 4.3 de l'étape 4 reproche à T17 (« T17 exclut le mécanisme même que la stratégie veut exploiter »), réintroduite dans la porte retenue. En l'état, la phase terminale est refusée presque intégralement.

Option	Conséquence
1. <code>D_max</code> devient un quantile mesuré en ligne	Cohérent avec R17, R18, R20. Mais perd son rôle de garde absolue contre une donnée aberrante
2. <code>D_max</code> sort de la conjonction et devient une pure alerte de journalisation	La phase terminale redevient accessible. La garde contre l'aberration passe à <code>Warden</code> sous un autre code
3. La phase [240;270[est abandonnée	La fenêtre d'action retombe à [180;240[, ce qui exclut à nouveau $\tau_{lock} = 262$ s et annule le bénéfice de R26

Ma recommandation : option 2. Une alerte journalisée conserve la détection sans stériliser la porte. L'option 3 annule le résultat le mieux étayé de l'étape 4.

D4. Quel est le plancher de décote réel ? (R37, R41)

Pourquoi c'est nécessaire. La table retient `δ = 0,02` mais déclare `δ_min = fee_sizing + 0,01 = 0,0207 à 0,0375` **non négociable**. Les deux ne peuvent pas être vrais. Et le premier multiple de tick admissible, 0,03, n'a **jamais eu son taux de contact mesuré** : les mesures existent à 0,02 (85,20 % et 91,76 %) et à 0,05 (70,92 %), et la chute entre les deux est nette.

Option	Conséquence
1. $\delta = 0,03$, plancher = <code>fee_sizing + 1 tick</code>	Conservateur. Taux de fill inconnu, à mesurer par M3. Marge par fill plus épaisse
2. Plancher = <code>fee_settlement + 1 tick ≈ 0,0132</code> , donc $\delta = 0,02$ admissible	Cohérent avec l'objection de l'étape 5 sur le biais de comparaison (§8.3) : pénaliser l'entrée d'un aller-retour alors que la position va au règlement est un biais

Ma recommandation : option 2, avec `fee_sizing` conservé uniquement pour le calcul de `N` (son rôle d'origine). C'est l'étape 5 qui a raison ici, et l'étape 4 qui traine une incohérence héritée du PDF. **Mais M3 doit balayer les deux** de toute façon, donc le coût de se tromper est faible.

D5. Quel délai d'invalidation pour une position remplie dont le côté ancré s'inverse ? (R48)

Pourquoi c'est nécessaire. L'étape 4 traite le cas d'un **ordre en vol** (annulation au premier tick, coût nul). Aucune règle n'existe pour une **position remplie**. L'étape 5 propose `h = 20 à 30 s` de persistance, mais cette valeur repose sur **une seule session** (A_1450, concordance 41,7 % dans [60;120]) et **aucune session UP ne teste la symétrie**.

Option	Conséquence
1. Aucune sortie : la position va au règlement	Cohérent avec <code>SETTLE_ONLY</code> . Mais accepte pleinement le piège mesuré dans le corpus (oracle à +4,64 \$ à 240 s, effondrement de 22 \$ en 15 s, issue DOWN)
2. <code>h = 20 s</code> , sortie au bid si exécutable	Limite la perte. Coûte la friction de 0,044 à 0,047 par unité, et le risque de whipsaw
3. Journalisé seulement, avec PnL contrefactuel des deux branches sur les 100 premières sessions	Ne décide rien aujourd'hui, mais produit la donnée qui décidera

Ma recommandation : option 3 d'abord, puis 1 ou 2 sur données. C'est la seule question du lot où je pense qu'aucune réponse n'est défendable aujourd'hui, et où mesurer coûte moins cher que choisir.

B. Les sept demandes non bloquantes

#	Demande	Pourquoi	Bloque
D6	Les fichiers des étapes 1, 2 et 3 , le ETAP4.MD brut (9 988 lignes), le PDF 17 pages , la méthode d'achat , les journaux de sessions , les erreurs identifiées et les fragments de code de l'ancien bot	Aucun fragment de code n'a été reçu, donc le registre ne contient aucune erreur de code : uniquement des erreurs de spécification. Et l'étape 4 déclare les conclusions des étapes 2 et 3 livrées vides , donc reconstruites depuis le PDF. Sans ces fichiers, je ne peux ni vérifier les citations ni trancher R39	Étape 4 (R39) et l'exhaustivité du registre
D7	Confirmez que le tick vaut 0,01 et que la plateforme accepte les ordres post-only	L'étape 4 précise que le tick est inféré du BBO, pas un champ de schéma . Et l'option A de D1 est impossible sans post-only	Étape 10
D8	Confirmez que K officiel, endDate , la profondeur L2 et la réconciliation de position sont obtenables via l'API	Ce sont les 4 bloquants qui empêchent physiquement le bot de coter . L'étape 4 conclut : « Si K officiel indisponible : arrêt du projet en l'état »	Étapes 3, 6, 7 pour le mode live. Le dry run peut avancer sans
D9	Validez z_min = 1,0 comme valeur de départ PROVISOIRE	L'étape 4 la marque explicitement provisoire (9/12 de confiance) et prévoit un balayage sur {0 ; 0,5 ; 1,0 ; 1,5 ; 2,0} après 500 sessions	Rien. Paramètre de configuration
D10	Validez τ_lock = 262 s malgré n = 1 session	C'est la valeur mesurée, contre 240 s hérité du PDF. Mais elle repose sur une seule observation. Je la retiens par défaut	Rien

		car mesurée, et je la rends configurable	
D11	Confirmez le budget de mesure de M3 : ≤ 400 \$ d'exposition sur 100 sessions, à 5 parts par événement	C'est le seul mécanisme qui crée les 11 champs manquants. Perte maximale par événement $\approx 5 \times ask \approx 4,00$ \$, espérance attendue proche de zéro	Étape 14
D12	Langage et environnement. Je propose Python 3.12 avec typage strict, structures immuables, et zéro dépendance réseau hors de <code>broker.py</code>	Le budget de latence cible est < 200 ms aller-retour et l'annulation ≤ 500 ms : Python passe largement. Le calcul de décision est mesuré à < 1 ms puisque les grandeurs sont déjà en mémoire. Si vous voulez du Rust ou du Go, dites-le maintenant : l'arborescence change	Tout

C. Ce que je ne validerai pas, même si vous le demandez

Deux garde-fous que je considère non négociables, parce que les abandonner reproduirait mécaniquement l'échec des étapes 1 à 3 :

1. **Aucun proxy de strike.** Un écart de 12,25 \$ inverse le règlement d'une session du corpus. Coder un proxy « en attendant » produirait des résultats de backtest qui ne signifient rien, et pire, des résultats qui **semblent** signifier quelque chose.
2. **Aucune constante gelée là où une mesure en ligne est possible.** C'est l'erreur unique qui a stérilisé la totalité du travail précédent : $\sigma_{30} = 26,50$ \$ (facteur $\times 3,68$), saut d'ask $\leq 0,01$ (10 à 20 $\times$ trop serré), $D_{max} = 0,114$ (dépassé 60,8 % du temps), P_{max} (refus 198/198). Chaque fois, la constante venait d'un régime unique et refusait le marché normal.

Et une réserve qui vaut pour tout le projet, que je répéterai à chaque étape : **le corpus couvre une seule journée, à 6,9 % de volatilité annualisée implicite contre 40 à 60 % typiques pour BTC, avec 3 issues indépendantes.** Rien de ce qui est chiffré en PnL dans ces documents n'est extrapolable. Ce qui est solide est **structurel** : l'absence de latence oracle vers carnet (0/27), la fausseté de σ_{30} (29/29), la stérilité de la formule à 22 tests (19/19), l'exactitude de l'invariant (1025/1025), la permanence du basis (100 %), et le fait que l'issue se verrouille après la fermeture de la fenêtre d'entrée.

 **Demande d'accord explicite.** Répondez au minimum à **D1, D2, D3, D4, D5** et **D12**, et je commence le codage à l'étape 0 (journalisation et structures communes), dans l'ordre du plan, avec les tests écrits en même temps que le code. Si vous voulez que j'assume mes recommandations par défaut sur D1 à D5, dites simplement « pars sur tes recommandations » et je le fais, en journalisant chaque choix par défaut comme une hypothèse révisable dans `config/params.yaml`.